

Technische Universität Dresden
Faculty of Computer Science
Chair of Privacy and Security in Ubiquitous Computing

**A Comparative Analysis and
Lifecycle-Oriented Tool Support
for Threat Modeling in Cloud Environments
with Privacy and Sovereignty Considerations**

Bachelor Thesis

Author: Bechir Sayadi
Student ID: 5199777
1.Supervisor: Prof./Dr. Ivan Gudymenko
2.Supervisor: Dr. Stefan Köpsell
Program: B.Sc. Computer Science
Deadline: 25 August 2026

Abstract

Cloud environments are dynamic, distributed, and difficult to secure using static threat modeling approaches. This thesis presents a comparative analysis of existing threat modeling tools and frameworks in the context of cloud security, privacy, and digital sovereignty. A lifecycle-oriented view is adopted to evaluate how well current tools support continuous threat management. A Proof-of-Concept system is developed to demonstrate practical, structured threat handling including identification, categorization, prioritization, and mitigation.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Research Goals	2
1.4 Research Questions	2
1.5 Thesis Structure	2
2 Background and Foundations	3
2.1 Cloud Computing and Architectures	3
2.1.1 Service Models: IaaS, PaaS, and SaaS	3
2.1.2 Cloud-Native Technologies	4
2.1.3 The Shared Responsibility Model	5
2.2 Threat Modeling Fundamentals	6
2.2.1 STRIDE	6
2.2.2 Attack Trees	8
2.2.3 Risk Assessment	8
2.3 Privacy Threat Modeling	9
2.3.1 LINDDUN	9
2.3.2 GDPR Relevance	10
2.4 Digital Sovereignty	10
2.4.1 Data Residency and Jurisdiction	10
2.4.2 EU Sovereignty Initiatives: Gaia-X	11
2.5 Security Benchmarks and Standards	11
2.5.1 NIST Cybersecurity Framework and CIS Benchmarks	12
2.5.2 ISO/IEC 27001 and CSA Cloud Controls Matrix	12
2.6 Chapter Summary	13
3 Related Work	14
3.1 Existing Threat Modeling Tools	14

3.1.1	Microsoft Threat Modeling Tool	14
3.1.2	OWASP Threat Dragon	15
3.1.3	IriusRisk	15
3.1.4	pytm and Code-Based Approaches	16
3.1.5	Tool Comparison Summary	17
3.2	LLM-Based Approaches	17
3.2.1	Automated Threat Generation	18
3.2.2	Privacy Threat Modeling with LLMs	19
3.2.3	Cloud-Native Attack Graph Generation	19
3.3	Research Gap	20
3.3.1	Positioning of This Thesis	22
3.4	Chapter Summary	23
4	Methodology	24
4.1	Research Design	24
4.2	Threat Lifecycle Model	24
4.3	Evaluation Criteria	25
4.4	Tool Selection Criteria	26
4.5	Proof-of-Concept Design Principles	27
4.6	Limitations of the Methodology	28
4.7	Chapter Summary	28
5	Comparative Analysis	29
5.1	Comparative Analysis Matrix	29
5.2	Analysis by Dimension	30
5.2.1	Cloud Support	30
5.2.2	Privacy and Sovereignty	31
5.2.3	Lifecycle Completeness	32
5.2.4	Automation and LLM Integration	33
5.3	Summary of Findings	33
5.4	Chapter Summary	34
6	Privacy and Sovereignty in Cloud Threat Modeling	35
6.1	Integrating Privacy Threats into Cloud Threat Models	35
6.1.1	The Distinction Between Security and Privacy	35
6.1.2	Applying LINDDUN in Cloud Environments	36
6.2	Sovereignty Threat Modeling	37
6.2.1	Defining Sovereignty Threats	37
6.3	Combined Threat Taxonomy	38
6.3.1	Worked Example: Applying the Taxonomy to a Single Data Flow	40

6.4	Chapter Summary	40
7	Proof-of-Concept Implementation	42
7.1	Requirements	42
7.2	Technology Stack	43
7.3	System Architecture	44
7.4	LLM Prompting Strategy	45
7.5	Example Use Case	46
7.5.1	System Under Analysis	46
7.5.2	Results	46
7.5.3	Key Finding	48
7.6	Limitations of the PoC	49
7.7	Chapter Summary	50
8	Evaluation and Discussion	51
8.1	PoC Requirements Evaluation	51
8.2	Research Questions Revisited	53
8.3	Recommended Workflow	53
8.4	Limitations	55
8.5	Chapter Summary	56
9	Conclusion and Future Work	57
9.1	Summary of Contributions	57
9.2	Answers to Research Questions	58
9.3	Future Work	58
9.4	Closing Remarks	59
	Bibliography	60

List of Figures

2.1	Cloud service models pyramid: IaaS, PaaS, and SaaS with provider vs. customer responsibility boundaries. Adapted from NIST SP 800-145 [1]. . .	4
2.2	Cloud-native architecture: code pushed by a developer flows through a CI/CD pipeline into a container registry, then into a Kubernetes cluster running microservices [5].	5
2.3	STRIDE applied to a Data Flow Diagram. Each element is annotated with applicable threat categories. Based on the Microsoft Threat Modeling Tool methodology [11].	7
2.4	Gaia-X federated cloud architecture showing the trust framework connecting providers, consumers, and the federation services [21].	11
2.5	NIST Cybersecurity Framework core functions: Identify, Protect, Detect, Respond, Recover (Version 1.1) [22]. Version 2.0 (2024) adds a sixth function, Govern, not shown here.	12
3.1	Simplified pipeline for LLM-based threat generation: a natural language system description is passed to an LLM, which produces a STRIDE-based threat list. The process is fast and flexible but carries a risk of non-deterministic or hallucinated output.	19
3.2	Overlap between security, privacy, and digital sovereignty in cloud threat modeling. Existing tools cover security well and privacy partially, but no tool addresses sovereignty, leaving the intersection of all three areas largely unaddressed. This gap is the focus of the combined taxonomy proposed in Chapter 6.	22
4.1	The four-stage threat lifecycle model, drawn as a circle rather than a straight line since the process starts over whenever the architecture changes in a meaningful way.	25
5.1	Lifecycle completeness across the five evaluated tools. Each axis is one lifecycle stage, scored N/P/Y as in Table 5.1. IriusRisk is the only tool that reaches the outer edge on every axis.	32

6.1	STRIDE and LINDDUN applied to the same simplified Data Flow Diagram. Red markers show STRIDE (security) threats, green markers show LINDDUN (privacy) threats. The flow between frontend and backend, for example, carries both a Tampering threat and a Linkability threat.	37
7.1	PoC pipeline architecture. A single LLM call performs both Stage 1 (Identification) and Stage 2 (Categorisation), since the model returns each threat's category and sub-category directly. Stages 3–4 are deterministic Python code, dashed to distinguish them from the single non-deterministic step. . .	44
7.2	PoC terminal output for the example AWS application, generated directly by <code>threat_analyzer.py</code> . The sovereignty threats (Jurisdictional Exposure, Data Residency Violation) are findings no evaluated tool from Chapter 5 would produce.	48
8.1	Recommended cloud threat modeling workflow, shown as a continuous cycle rather than a straight line, matching the lifecycle model introduced in Chapter 4.	55

List of Tables

2.1	STRIDE threat categories with violated security properties and examples.	7
2.2	LINDDUN privacy threat categories.	9
3.1	Summary comparison of reviewed threat modeling tools.	17
3.2	Summary of key LLM-based threat modeling approaches.	20
4.1	Threat lifecycle stages used as the evaluation framework.	25
4.2	Evaluation criteria for the comparative analysis.	26
5.1	Full comparative analysis matrix. Y = Yes, P = Partial, N = No.	30
6.1	Combined security-privacy-sovereignty threat taxonomy for cloud environments.	39
7.1	PoC functional and non-functional requirements.	43
7.2	PoC output threats for the example AWS system, sorted by risk score. . .	47
8.1	PoC requirements fulfilment assessment.	52
8.2	Research questions and corresponding answers from thesis findings.	53

Chapter 1

Introduction

1.1 Motivation

Cloud computing changed how companies build and operate their systems. It is flexible, can scale quite easy, and in many situations it costs less than having their own servers. But because of this flexibility, security is also more difficult. The traditional threat modeling approach, where one diagram is created once, is often not enough for cloud environments.

Cloud environments are changing all the time. New servers can be created and removed automatically, microservices communicate over different network paths, and CI/CD pipelines also change what services are reachable from outside. Therefore, threat modeling for cloud systems needs to consider identity and access problems, wrong infrastructure configurations, supply-chain attacks, and also that cloud-native applications are changing continuously [1].

Another important topic is privacy and data sovereignty. GDPR made privacy threat modeling necessary for many organizations in Europe. Also, if a cloud provider is located outside the EU, there can be questions about data sovereignty. For example, where the data is really stored, which country's law is used, or if another government can require the provider to give access to the data, like under the US CLOUD Act [2], [3].

1.2 Problem Statement

Even if cloud threat modeling becomes more important, the available methods are still quite separated and usually they are not used together. STRIDE is one of the most common frameworks for security threats, but it does not really include privacy or sovereignty topics. LINDDUN focuses on privacy, however in many cases it is applied separately from normal security analysis. For sovereignty, I could not find a tool that covers it in a complete way.

Another recent direction is using LLMs to support threat identification automatically. This idea looks useful, but there are still some problems. The generated results are not

always consistent and sometimes the model can produce information which is not correct. Also, these approaches do not consider sovereignty issues. Based on the papers I reviewed, I did not find a detailed comparison that evaluates existing approaches for security, privacy, and sovereignty at the same time.

1.3 Research Goals

The main goal of this thesis is to investigate how threat modeling can be supported during the whole lifecycle of cloud environments. It also checks if existing tools and LLM-based approaches are able to identify security, privacy and sovereignty threats in a useful way.

Another goal is to design a practical workflow which can be used in actual use and test it with a small Proof-of-Concept system. The idea is that a user provides a description of a cloud architecture, then the system generates a threat report. The report should include threats from all three areas.

1.4 Research Questions

1. Which cloud security benchmarks matter most for threat modeling?
2. How well do the threat modeling tools out there cover the whole lifecycle, from spotting threats to fixing and reporting them?
3. What do LLM-based threat libraries actually add when it comes to finding security and privacy threats in cloud systems?
4. How can sovereignty threats be built into cloud threat models alongside the usual security and privacy threats?
5. Which tool, or combination of tools, works best for keeping up a continuous cloud threat modeling process?

1.5 Thesis Structure

Chapter 2 covers the needed background: cloud architectures, STRIDE, LINDDUN, digital sovereignty, and security benchmarks. Chapter 3 looks at existing tools and LLM-based approaches and points out where the gaps are. Chapter 4 explains how I set up the evaluation, the lifecycle model I used, and how I designed the PoC. Chapter 5 walks through the tool comparison. Chapter 6 lays out the combined threat taxonomy. Chapter 7 describes how the PoC was built and what it found. Chapter 8 looks back at the results and suggests a workflow. Chapter 9 wraps everything up and points to future work.

Chapter 2

Background and Foundations

This chapter introduces the background concepts that are required for the rest of this thesis. It first explains the basic cloud computing architectures and deployment models because they influence how security threats can appear in cloud environments. Afterwards, common threat modeling frameworks such as STRIDE for security and LINDDUN for privacy are presented. The chapter also discusses digital sovereignty, including aspects like data residency and legal regulations, and summarizes security benchmarks such as the CIS Benchmarks and NIST guidelines that are relevant for evaluating cloud systems. These concepts provide the basis for the comparison, taxonomy, and the Proof-of-Concept presented in the following chapters.

2.1 Cloud Computing and Architectures

2.1.1 Service Models: IaaS, PaaS, and SaaS

Cloud computing means that computing resources are provided over the Internet instead of running everything on local servers. Users can rent storage, computing power, and software when they need them and usually only pay for the resources they use. According to NIST SP 800-145, cloud computing is commonly divided into three main service models [1].

Infrastructure as a Service (IaaS) provides virtual infrastructure such as virtual machines, storage, and networking resources. In this model, the cloud provider manages the physical hardware and virtualization, while the customer is still responsible for the operating system, installed software, and applications. Typical examples are Amazon EC2, Google Compute Engine, and Azure Virtual Machines.

Platform as a Service (PaaS) offers an environment where applications can be deployed without managing the underlying operating system. The provider takes care of the runtime, middleware, and system updates, while the customer mainly focuses on developing and deploying the application. Google App Engine, Azure App Service, and Heroku are common examples.

Software as a Service (SaaS) provides complete software applications that users can access directly through the Internet. In this model, almost all technical management is done by the cloud provider, while the customer mainly uses the application and manages its own data or user accounts. Examples include Microsoft 365, Google Workspace, and Salesforce.

These service models are important for threat modeling because they define the *security boundary* between the cloud provider and the customer. Depending on the service model, the customer has different responsibilities. For example, in IaaS the customer must secure the operating system and applications, while in SaaS most of the infrastructure is managed by the provider. Therefore, the service model affects which threats are relevant and which security controls can be applied by the customer [4].

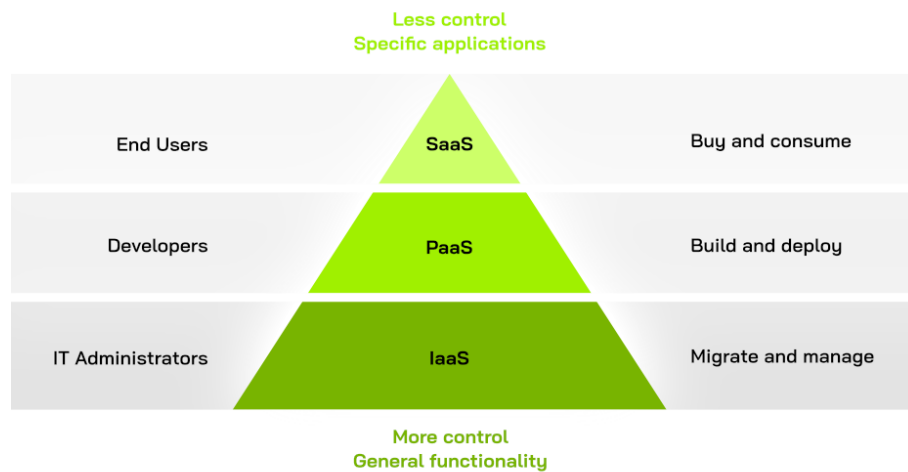


Figure 2.1: Cloud service models pyramid: IaaS, PaaS, and SaaS with provider vs. customer responsibility boundaries. Adapted from NIST SP 800-145 [1].

Image source: <https://www.optimal-systems.de/wp-content/uploads/2023/12/cloud-service-models-pyramid-blog-en.webp>

2.1.2 Cloud-Native Technologies

Modern cloud applications are different from traditional software because they are built for distributed environments and need to support scaling and frequent changes [5].

Containers package an application together with its required libraries and dependencies so that it can run in different environments. Docker is one of the most used container platforms. Compared to virtual machines, containers are more lightweight because they share the host operating system kernel and usually start much faster.

Kubernetes is widely used to deploy and manage containers in cloud environments. It provides functions such as automatic deployment, scaling and monitoring of applications. However, Kubernetes also introduces several security risks. For example, attackers may

target the API server, the etcd database, service accounts or secrets if they are not configured correctly [6].

Microservices divide an application into several smaller services that communicate over network APIs. This architecture makes applications easier to scale and update, but it also creates more communication paths between services. Because of this, more trust boundaries have to be considered during threat modeling.

CI/CD pipelines automate software building, testing and deployment. Common tools are GitHub Actions, GitLab CI and Jenkins. Since these pipelines often have high privileges, they can become an important attack target. If an attacker gets access to the pipeline, malicious code or compromised dependencies can be deployed to production, which is a typical software supply chain attack [7].

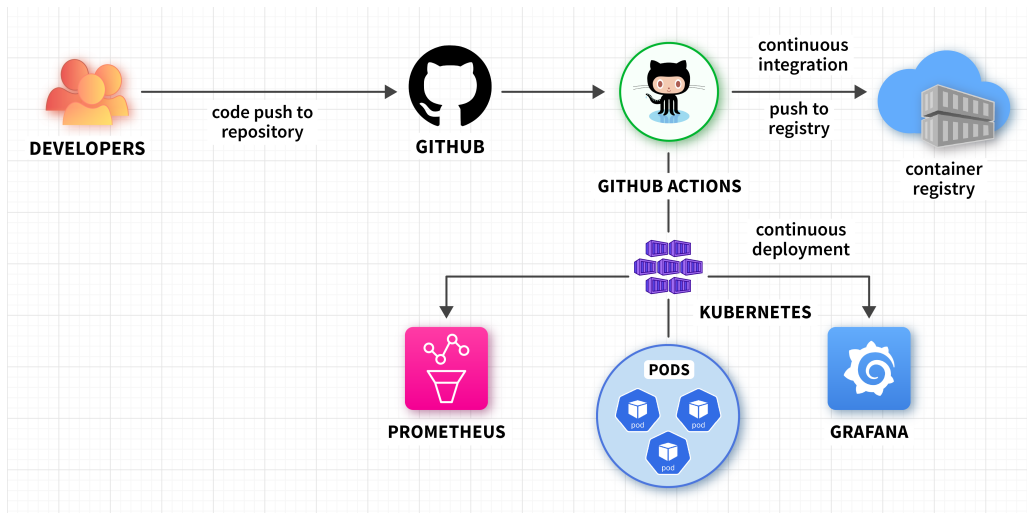


Figure 2.2: Cloud-native architecture: code pushed by a developer flows through a CI/CD pipeline into a container registry, then into a Kubernetes cluster running microservices [5].

Image source:

<https://dzone.com/articles/building-a-cicd-pipeline-with-kubernetes>,
<https://dz2cdn1.dzone.com/storage/temp/17952360-nvootukuri-figure-1.png>

2.1.3 The Shared Responsibility Model

One important concept in cloud security is the *shared responsibility model*. It describes how security tasks are divided between the cloud provider and the customer. The provider is responsible for protecting the cloud infrastructure itself, while the customer is responsible for securing the systems and applications that are deployed inside the cloud environment [8].

The exact responsibilities depend on the used cloud service model. In general, the provider manages parts such as the physical data centres, hardware, virtualisation layer, and the basic network infrastructure. The customer still has to take care of areas such as identity and access management (IAM), encryption settings, application security, and

correct configuration of the deployed resources.

A large number of cloud security problems are caused by incorrect configurations or missing security settings. For example, a publicly accessible storage bucket or IAM permissions that are too broad can allow attackers to access sensitive resources. In such cases, the cloud provider infrastructure can still be secure, but the customer configuration creates a security risk. An example for this problem is the 2019 Capital One data breach, where customer information was exposed due to a vulnerable configuration of an AWS Web Application Firewall [9].

Therefore, the responsibility boundaries are important when defining the scope of a threat model. In an IaaS environment, the threat model should include customer-managed components such as the operating system, applications, and cloud configurations. In comparison, SaaS environments provide less control to the customer, so the focus is more on aspects like data handling, user access control, and configuration of the provided service.

2.2 Threat Modeling Fundamentals

Threat modeling is a method to identify possible security problems of a system and define suitable countermeasures before these problems can be exploited by attackers. Instead of reacting to vulnerabilities after deployment, threat modeling tries to consider security risks already during the design and development phases. One well-known approach from Microsoft describes this process using four main questions: What are we building? What can go wrong? What are we doing about it? Did we do a good enough job? [10]

2.2.1 STRIDE

STRIDE is one of the commonly used approaches for threat classification in threat modeling. It was introduced by Microsoft in the late 1990s as part of their threat modeling methodology [11]. The approach divides possible threats into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Each category describes a different type of security violation, for example when an attacker pretends to be another user or modifies information without authorization.

Table 2.1: STRIDE threat categories with violated security properties and examples.

Letter	Threat	Violated Property	Example
S	Spoofing	Authentication	Attacker pretends to be a real user
T	Tampering	Integrity	Data gets changed in transit
R	Repudiation	Non-repudiation	User denies doing something
I	Information Disclosure	Confidentiality	Sensitive data leaks from a database
D	Denial of Service	Availability	Service gets flooded with requests
E	Elevation of Privilege	Authorisation	User gets admin rights they shouldn't have

STRIDE is applied on a *Data Flow Diagram* (DFD) that represents the system architecture which is analysed. The different elements of the DFD, such as processes, data stores, data flows, and external entities, are examined according to the STRIDE categories. For each element, possible security threats are identified, for example spoofing, tampering, or information disclosure. The result is a list of potential threats that can later be analysed and addressed with suitable mitigation measures [10].

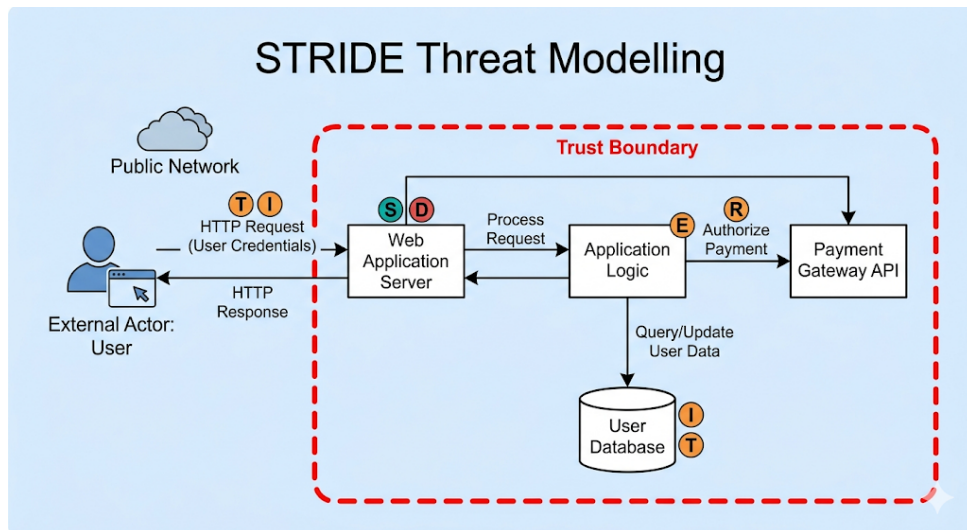


Figure 2.3: STRIDE applied to a Data Flow Diagram. Each element is annotated with applicable threat categories. Based on the Microsoft Threat Modeling Tool methodology [11].

Image source: https://miro.medium.com/1*9g-HIUcUUzo1bgKDvoxAYQ.png

2.2.2 Attack Trees

Attack trees were introduced by Bruce Schneier in 1999 as a method to describe possible attack paths against a system [12]. The main goal of an attack is placed at the root node of the tree, for example gaining unauthorized access to a database. The lower nodes describe possible steps that an attacker needs to perform in order to reach this goal. These steps can be further divided into smaller actions until a more detailed attack path is created.

Attack trees can be combined with STRIDE because both approaches describe security threats from different perspectives. STRIDE is mainly used to categorize possible threats, for example spoofing or information disclosure, while attack trees focus more on the sequence of actions that an attacker could perform. Therefore, attack trees can help to understand how several weaknesses could be combined to perform a larger attack. This is useful in cloud environments where attacks can involve multiple components such as APIs, identities, and different cloud services.

Another related concept is the *attack surface*. It describes all possible entry points where an attacker can interact with a system or try to access data. Cloud environments usually have a larger attack surface because they contain many externally reachable services, APIs, user interfaces, and configuration options. For this reason, new cloud resources and services can introduce additional risks if they are not correctly secured [13].

2.2.3 Risk Assessment

Finding threats alone is not sufficient, because different threats have different levels of importance. Therefore, it is necessary to evaluate and prioritize them before selecting mitigation measures. Risk assessment helps with this process by considering two main factors for each identified threat: the likelihood that the threat can occur and the possible impact if it happens [14].

- **Likelihood:** how likely is this threat to actually happen?
- **Impact:** how bad would it be if it did?

The risk score can be calculated by combining the likelihood of a threat and its possible impact: $\text{Risk} = \text{Likelihood} \times \text{Impact}$. Different approaches exist to make this calculation more structured. For example, CVSS (Common Vulnerability Scoring System) is widely used for evaluating the severity of known vulnerabilities [15]. It mainly focuses on technical vulnerabilities and uses different metrics such as attack complexity and required privileges. Another approach is DREAD (Damage, Reproducibility, Exploitability, Affected users, and Discoverability), which was introduced by Microsoft for risk estimation. However, DREAD has some problems because the scoring can depend strongly on the person performing the evaluation. In many practical threat modeling activities, a simple 3×3 risk matrix

with values such as low, medium, and high is still used because it is easier to apply and understand.

2.3 Privacy Threat Modeling

Security threat modeling mainly focuses on identifying possible attacks against a system and protecting the system assets from attackers. Privacy threat modeling has a different focus because it considers how the system itself can create privacy problems for the users whose data is processed. These problems do not always happen because of an attack, but can also result from design decisions of the system. For example, a system can be technically secure but still have privacy issues if it collects unnecessary personal data, stores information for too long, or uses collected data for purposes that were not expected by the users.

2.3.1 LINDDUN

LINDDUN is a privacy threat modeling framework developed by researchers at KU Leuven. It can be used together with security threat modeling approaches such as STRIDE, because both approaches can use the same Data Flow Diagram as input. While STRIDE focuses on security threats, LINDDUN analyzes the system components and data flows from a privacy perspective to identify possible privacy risks [16]. The framework defines different privacy threat categories, such as linkability, identifiability, and information disclosure, which help to identify privacy problems during the system design phase.

Table 2.2: LINDDUN privacy threat categories.

Letter	Threat	Description
L	Linkability	Linking data or actions back to a person
I	Identifiability	Figuring out who a person is from the data
N	Non-repudiation	Person can't deny doing something
D	Detectability	Telling that data about someone exists
D	Disclosure of Info.	Exposing someone's personal data
U	Unawareness	Person has no idea their data is being processed
N	Non-compliance	Breaking a regulation or policy

LINDDUN is relevant for this thesis because it provides a structured approach for identifying privacy-related threats in a system. Compared to general security threat modeling, LINDDUN focuses on privacy aspects and can support the analysis of GDPR-related requirements, such as privacy by design and data protection principles. Recent research also investigated the combination of LINDDUN and LLMs for supporting the

automatic identification of privacy threats [17]. This approach is interesting for this thesis because LLMs could help to reduce the manual effort required during privacy threat modeling.

2.3.2 GDPR Relevance

GDPR has been applicable since May 2018 and represents the main legal framework for processing personal data in the European Union [2]. For this thesis, GDPR is important mainly because it defines requirements for protecting personal data and influences how cloud systems should be designed.

One important aspect is Article 25, which introduces the concept of *privacy by design and by default*. This means that privacy considerations should already be included during the development and configuration of a system, instead of being added only after security or privacy problems appear. In this context, privacy threat modeling approaches such as LINDDUN can support the identification of possible privacy risks during the design phase.

Another relevant part is Chapter V of GDPR, which regulates the transfer of personal data to countries outside the EU. When cloud services process data in third countries, additional safeguards such as Standard Contractual Clauses (SCCs) may be necessary. Therefore, GDPR requirements are also connected with the question of where data is stored and which legal environment applies to it. This connection between privacy protection and data sovereignty is further discussed in Section 2.4 [18].

2.4 Digital Sovereignty

Digital sovereignty is a relatively new concept in European policy and technology. It concerns whether individuals, companies, and states can actually control their own data, infrastructure, and technology choices, without foreign influence or being forced to hand things over.

2.4.1 Data Residency and Jurisdiction

Data residency refers to where data physically resides. *Data jurisdiction* refers to which country’s laws apply to it. In the cloud, these two concepts often do not align: data stored on a server in Germany can still fall under US law if the provider is a US company.

This mismatch creates a real legal and security risk. The US CLOUD Act (2018) allows US authorities to compel cloud providers to hand over data regardless of where it is physically stored [3]. The 2020 Schrems II ruling by the Court of Justice of the EU addressed the same tension from the other direction, invalidating the EU–US Privacy Shield framework because US surveillance law did not offer EU data subjects protections equivalent to the GDPR [19]. As a result, European companies using US-based providers

such as AWS, Azure, or Google Cloud remain exposed to potential foreign government access, even if their data never leaves European data centres [20].

2.4.2 EU Sovereignty Initiatives: Gaia-X

Gaia-X is Europe’s largest initiative aimed at addressing cloud sovereignty. The goal is to build a federated, open, and transparent cloud ecosystem that follows European values and regulations, with data sovereignty and interoperability built in from the start [21].

Gaia-X has a Trust Framework that sets out requirements such as data portability, transparency regarding sub-processors, and compliance with European law. These requirements map fairly directly onto real threat mitigations: for example, the portability requirement helps against vendor lock-in, while sub-processor transparency helps address the problem of not knowing where data actually ends up.

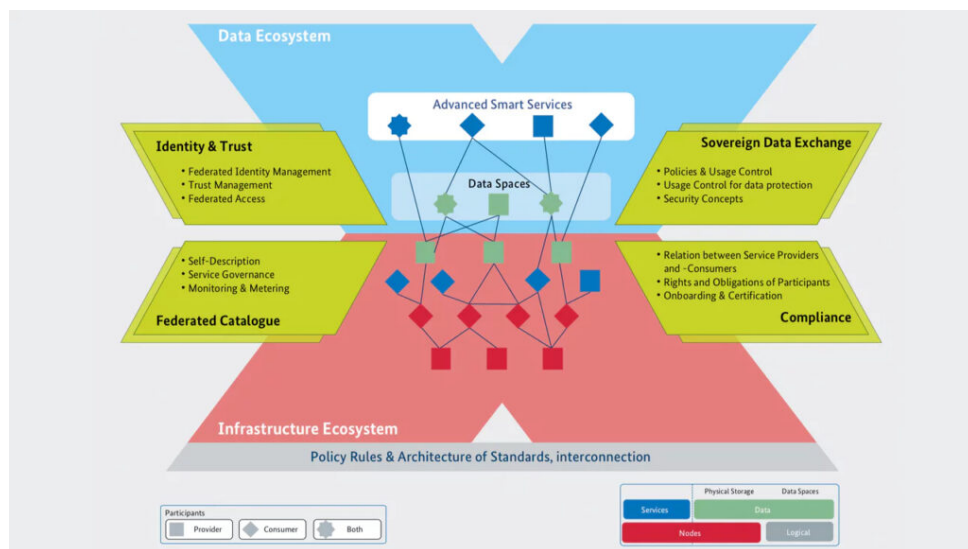


Figure 2.4: Gaia-X federated cloud architecture showing the trust framework connecting providers, consumers, and the federation services [21].

Image source: <https://lamarr-institute.org/wp-content/uploads//Abbildung-1-Architekturueberblick-1024x576.jpg>

2.5 Security Benchmarks and Standards

Security benchmarks and standards give organisations concrete, actionable controls. They matter for threat modeling because they essentially form the toolbox of fixes: every threat that is found should map to one or more controls from these frameworks.

2.5.1 NIST Cybersecurity Framework and CIS Benchmarks

The NIST Cybersecurity Framework (CSF) organises everything into five core functions: Identify, Protect, Detect, Respond, and Recover [22]. It is technology agnostic and is used widely by both public and private organisations. Version 2.0, released in 2024, added a sixth function, Govern, which focuses on accountability.

CIS Benchmarks provide detailed, specific configuration guidelines for particular technologies, with dedicated benchmarks for AWS, Azure, GCP, Kubernetes, Docker, and more [23]. Each benchmark gives testable controls, for example making sure S3 bucket server-side encryption is turned on, so they are directly usable in cloud environments. They are also free, and are among the most practically useful resources available for cloud security configuration.



Figure 2.5: NIST Cybersecurity Framework core functions: Identify, Protect, Detect, Respond, Recover (Version 1.1) [22]. Version 2.0 (2024) adds a sixth function, Govern, not shown here.

Image source: https://www.nist.gov/sites/default/files/styles/220_x_220_limit/public/images/2018/04/12/ipdrr_circle.png?itok=qV5agiH5

2.5.2 ISO/IEC 27001 and CSA Cloud Controls Matrix

ISO/IEC 27001 is the international standard for Information Security Management Systems (ISMS). It sets out requirements for establishing, operating, and continuously improving how an organisation manages sensitive information [24]. Annex A contains 93 security controls organised into four themes. ISO 27001 certification is commonly required in enterprise procurement and serves as proof of a solid security baseline.

The CSA Cloud Controls Matrix (CCM) is built specifically for cloud environments. It maps cloud-specific controls to major frameworks such as ISO 27001, NIST, and GDPR, and is organised into 17 domains covering areas such as identity and access management,

infrastructure security, supply chain management, and data security [25]. Its explicit coverage of data sovereignty and residency makes it especially relevant to this thesis.

2.6 Chapter Summary

The main points of this chapter can be summarised as follows:

- Cloud environments have a huge, constantly changing attack surface that old-fashioned, one-time threat modeling was never built to handle.
- The shared responsibility model draws a critical line between what the provider secures and what the customer secures, and misunderstanding that line is the most common cause of cloud security failures.
- STRIDE and LINDDUN complement each other: together they cover security and privacy threats, and both can be applied to the same Data Flow Diagram.
- Sovereignty threats form an entirely separate dimension. They arise from legal and geopolitical context rather than technical bugs, and neither STRIDE nor LINDDUN addresses this. Current tools mostly ignore it as well.
- Security benchmarks and standards (NIST, CIS, ISO 27001, CSA CCM) provide the vocabulary of mitigations used later in the comparison and the PoC.

These points set up the comparison in Chapter 5 and the combined threat taxonomy in Chapter 6.

Chapter 3

Related Work

This chapter reviews different existing threat modeling tools and approaches based on LLMs that are related to cloud security. The purpose is to understand not only what functions these tools provide, but also what limitations they have when they are used in cloud-native environments. The chapter also discusses why some problems still remain unsolved and identifies the research gap that is considered in the following parts of this thesis.

3.1 Existing Threat Modeling Tools

Most threat modeling tools assume something that used to be reasonable: that a system can be drawn as a diagram, and that diagram stays correct long enough to be useful. That assumption worked fine for the client-server and n-tier systems STRIDE was built for in the late 1990s. It does not really hold anymore. A cloud deployment can look different after every pipeline run, so a diagram from last month might already be wrong. The four tools reviewed below deal with this in quite different ways, and I think those differences say more about each tool's philosophy than any feature list would.

3.1.1 Microsoft Threat Modeling Tool

The Microsoft Threat Modeling Tool (TMT) is one of the older and also one of the more commonly used threat modeling tools. It uses STRIDE methodology with a graphical Data Flow Diagram editor, where the user creates components, connections, and trust boundaries. After this, the tool checks the different elements of the diagram and generates possible threats based on predefined rules [11]. The tool is free and has a lot of documentation. It is also connected with Microsoft's Security Development Lifecycle, which is probably one reason why many organisations using Microsoft technologies still use it.

The core problem is that a DFD is static by nature. In TMT, a "Web Server" component is just a process with known inputs and outputs, fixed once you draw it. Cloud-native

systems don't really work like that. An EKS pod might live for a few minutes before it's rescheduled onto a different node with a different IP. A Lambda function exists only while it's running and then it's gone. None of this has a place in TMT's model ; there's no concept for IAM roles, autoscaling groups, or service mesh sidecars. So users end up forcing these things into generic "process" or "data store" boxes, and whatever made them cloud-specific in the first place gets lost in translation.

TMT also just doesn't look at privacy or sovereignty at all ; it was never built to. Combined with the fact that it hasn't seen many updates lately [11], my guess is that Microsoft has more or less stopped investing in it. For a classic on-premises Microsoft shop that might not matter much. For a cloud-native team, though, it means the threats the tool produces can end up describing a system that doesn't really exist anymore.

3.1.2 OWASP Threat Dragon

OWASP Threat Dragon uses the same DFD and STRIDE based approach as TMT, but it is provided as an open-source community maintained tool which can be used as a web application or desktop application. The threat models are stored in JSON format, which is useful because the model can be placed inside the same Git repository with the related source code. Like this, changes of the threat model can be reviewed in pull requests together with code changes instead of being a separate document which can be forgotten after some time [26]. Compared with TMT, this makes Threat Dragon more connected with the normal software development process and not only something done before a release for compliance reasons.

The scope is narrower, though. Threat Dragon is really built to make threat identification approachable for people without a security background, and that focus comes at a cost. There's no integrated risk scoring, no mitigation management, no structured reporting. A team can find threats with it, fine ; but deciding what to prioritize, tracking remediation, producing something an auditor would accept, all of that needs separate tooling. For a volunteer-run open-source project that's probably a fair trade-off given limited resources. It still means Threat Dragon covers less of the lifecycle than the other tools reviewed here, and like TMT it has no cloud-specific libraries, no privacy modeling, and no sense of sovereignty at all.

3.1.3 IriusRisk

IriusRisk is more interesting in this comparison because it is the only tool in this review which seems to be designed also for the later stages of the lifecycle, not only for identifying threats. It supports different modeling methodologies and it can connect with Jira and GitHub, which means that generated threats can be converted into tickets that can be followed later instead of only staying as items in a list. It also provides pattern libraries

for AWS and Azure components which are more specific than only using generic DFD elements [27]. For example, when a component is defined as an S3 bucket in IriusRisk, it can include threats related to this specific service, such as public access configuration problems, missing server-side encryption or permissions which are too broad. This is not the same as a general threat like "Information Disclosure on Data Store" which could be generated by a DFD based tool such as TMT.

This support for the complete lifecycle is one of the main reasons why IriusRisk receives good results in Chapter 5. The open-source tools analysed in this work do not reach the same level mainly because reporting and tracking mitigation activities require a lot of engineering effort. These parts are usually less visible in demonstrations and they are also not the main topic in many threat modeling research papers. They need continuous development and maintenance, not only a good algorithm. IriusRisk has this investment because it is a commercial tool, but this also creates a disadvantage. The license price which supports these features can make it difficult to use for a reproducible academic PoC or for a small development team. The tool also provides mappings for GDPR and NIST requirements, but these are mostly based on checking if controls exist, for example if there is a consent mechanism or a data retention policy. This is not the same as a LINDDUN analysis where specific data flows are examined for risks such as linkability or identifiability. Also, even if IriusRisk covers different compliance areas, it does not model sovereignty. The rules do not consider where a component is hosted, which jurisdiction applies, or which government authority could request access to the data.

3.1.4 pytm and Code-Based Approaches

Unlike the usual diagram-based approach of threat modeling, pytm describes the system using Python code instead of a diagram. Components, data flows, and trust boundaries are created as objects, and the threat generation is done by running a function on this object structure rather than checking a drawing manually [28]. This can be useful because threat modeling is not only done one time in a design meeting, but it can also be included in a CI/CD process and used to detect problems before deployment, similar to how tests or code checking tools are used.

This fits cloud-native development reasonably well, since most cloud systems already exist as code anyway ; Terraform, CloudFormation, Kubernetes manifests. In theory, going from infrastructure-as-code to a pytm model should be a smaller step than starting a diagram from scratch. In practice, pytm doesn't read Terraform or Helm files directly, so someone still has to translate the architecture into pytm's Python objects by hand. And that's the real barrier: you need to write and maintain Python code, which is not something every security person is comfortable doing. It's less approachable than TMT or Threat Dragon for that reason alone. Like the other two, it has no privacy modeling and

no sovereignty concept. It automates threat generation, sure, but the risk scoring is thin and there’s no built-in reporting, so ; same story as Threat Dragon ; you’re still reaching for other tools to finish the job.

3.1.5 Tool Comparison Summary

Table 3.1 summarises the four tools discussed above, plus two additional code-based and hybrid tools ‘ThreatSpec [29] and CAIRIS [30]’ that were reviewed during tool selection but not carried through to the full comparative analysis in Chapter 5, since Section 4.4’s selection criteria required only one representative of each tooling style.

Table 3.1: Summary comparison of reviewed threat modeling tools.

Tool	Type	Method	Cloud	Open Source	Automation
MS TMT	Desktop	STRIDE/DFD	Partial	No	Low
Threat Dragon	Web/Desktop	STRIDE/DFD	Partial	Yes	Low
IriusRisk	Platform	STRIDE + others	Yes	No	Medium
pytm	Code-based	STRIDE	Yes	Yes	High
ThreatSpec	Code-based	Custom	Yes	Yes	High
CAIRIS	Platform	Multiple	Partial	Yes	Medium

The table does not show another important point. The four tools are divided in a quite clear way. The diagram-based tools (TMT, Threat Dragon) are still more similar to the original STRIDE approach, but this also creates problems when they are used for cloud-native systems. The code-based and platform-based tools (IriusRisk, pytm) changed the original DFD model more, but they changed it in different ways. IriusRisk focuses more on workflow integration and covering more parts of the lifecycle, while pytm is more connected with automation and CI/CD usage. However, none of these tools added privacy or sovereignty aspects. This is probably because STRIDE itself does not include concepts for these topics, and the reviewed tools still mainly consider the six STRIDE categories as the main definition of what can be a threat.

3.2 LLM-Based Approaches

Recent studies have investigated the use of Large Language Models, such as GPT-4, Claude, and open-source models like Llama, for helping with or automating some parts of the threat modeling process. Compared with the four tools mentioned above, these approaches are not one specific tool with defined functions. Instead, they include different research works that use similar LLM-based methods, but they have different goals and

methods. There is also still no clear agreement about what kind of threat information generated by an LLM should represent or how this information should be used.

3.2.1 Automated Threat Generation

The most direct application is to replace the DFD-plus-rule-engine pipeline with one step: the analyst writes a description of the system in normal language, and the model identifies possible threats directly using STRIDE or another threat taxonomy [31]. This can avoid the problem of maintaining diagrams, which is one issue with tools like TMT and Threat Dragon especially in cloud environments. There is no diagram that becomes old, because no diagram was created. However, it is not completely clear if this is always better. A DFD makes the analyst define trust boundaries and data flows more clearly, while a text description depends on what information is included by the analyst. The LLM cannot find something which is not described in the input. For example, if the description does not mention that a database has a public read replica, the generated threats will probably not include this case. With a DFD, this component may be visible and the analyst could notice the issue earlier.

One advantage mentioned for LLM-based approaches is that they can understand more context and identify threats that are not already included in fixed rules. A traditional rule engine can only detect cases that were already considered when the rules were created. However, there are also important problems. The first one is non-determinism, because the same input may produce different threat lists in different executions. This can create problems for auditability and for processes where the same result should be reproduced. The second problem is hallucination. The model may create a threat that looks reasonable, uses the correct STRIDE category, and explains a possible attack, but the component or system behaviour may not actually match the assumption behind the threat [32]. Because of these issues, this thesis does not consider LLM threat generation as a full replacement for rule engines. Instead, it is used as one part of a system with deterministic controls around it, which is discussed further in Chapter 7.

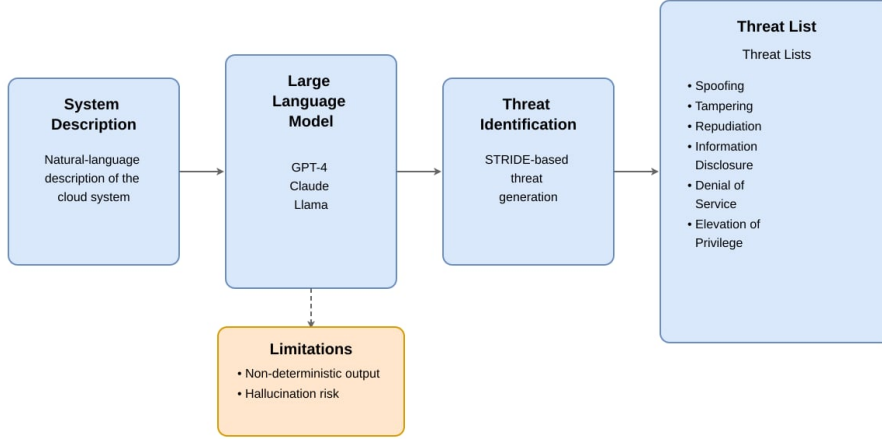


Figure 3.1: Simplified pipeline for LLM-based threat generation: a natural language system description is passed to an LLM, which produces a STRIDE-based threat list. The process is fast and flexible but carries a risk of non-deterministic or hallucinated output.

3.2.2 Privacy Threat Modeling with LLMs

Researchers at KU Leuven, the same research group that created LINDDUN, also studied how the LINDDUN methodology can be combined with LLM-based analysis. They showed that privacy threat assessments can be generated from system descriptions with much less effort compared to a traditional LINDDUN workshop [17]. This example is important because it is one of the cases where an AI tool is used for privacy analysis instead of only being discussed as a general risk. The main difference compared with a simple LLM prompt for threat generation is that the LINDDUN structure is still used. The model works with the seven existing privacy threat categories instead of creating new categories by itself. This gives more control to the analysis and avoids that the LLM produces only general privacy problems without following a specific method, which is also related to the discussion in Chapter 5 about the importance of having a structured LLM pipeline.

This work is also related to the taxonomy presented in Chapter 6 because it shows that LINDDUN analysis can be supported by automation while keeping the original LINDDUN structure. This is similar to the idea used in this thesis for sovereignty in Chapter 6 and the implementation approach shown in the PoC in Chapter 7.

3.2.3 Cloud-Native Attack Graph Generation

A separate research direction uses LLMs for creating attack graphs, which means complete multi-step attack paths in a system, from cloud architecture descriptions instead of only producing separate threat lists. Research about this specific use case is still limited. It is based on similar LLM capabilities for threat detection discussed before [31], but here the problem is more difficult because different issues need to be combined together. Attack graphs are useful because single STRIDE threats usually do not show the full attack

possibility. For example, an S3 bucket with public access and a Lambda function with too many IAM permissions are two different problems when they are checked separately. However, an attacker could use both together, for example by putting a malicious object in the exposed bucket and then using the Lambda function to process it with higher privileges. This type of attack path cannot be seen easily from the individual findings. Creating these chains manually in a medium or large cloud environment is very difficult, so automatic generation of attack graphs is an interesting approach.

However, this research area is still mainly based on prototypes and not widely used tools. Connection with the common security tools discussed in Section 3.1 is almost not existing, and similar to other topics in this chapter, the sovereignty aspect has not really been considered for attack graphs. In theory, an attack graph could also include changes between different jurisdictions, similar to how it shows movement between networks or systems. But currently there seems to be no work that has implemented this idea.

Table 3.2: Summary of key LLM-based threat modeling approaches.

Approach	Input	Output	Privacy	Limitation
LLM threat gen. (GPT-4)	Text description	Threat list (STRIDE)	Partial	Non-deterministic
LINDDUN-LLM hybrid	DFD + text	Privacy threats	Yes	Requires DFD
Attack graph generation	Cloud arch.	Attack paths	No	Prototype only
LLM prioritisation	Threat list	Ranked threats	Partial	Hallucination risk

3.3 Research Gap

Three main observations can be seen from the reviewed material above.

The first observation is that lifecycle completeness and openness have a conflict between them in the tools analyzed here. IriusRisk is the only tool which supports the whole process from identification until audit-ready reporting, but it is also the only commercial solution. Reporting features and mitigation tracking are usually less visible parts of threat modeling tools and they are not the main focus when developing open-source projects. This can happen because volunteer developers normally have limited time and these parts are more difficult to maintain. In comparison, a commercial company has more motivation to complete these functions because the product is connected with revenue. The acceptance of this limitation depends on the user situation: an enterprise with enough resources can pay for IriusRisk and use a more complete workflow, but a small team or an academic project that needs reproducibility usually cannot. For this reason, they need to select between open-source tools which often provide good threat identification but have fewer features after this stage.

The second point is that sovereignty is not a dimension which is partly considered by

some tools and missing in others. In the reviewed tools and LLM-based approaches, it is not covered at all. This result is not only because sovereignty is a new topic. STRIDE was created in 1999, when cloud computing was not developed as it is today, and also before regulations such as the CLOUD Act and Schrems II made data sovereignty more important from a legal perspective. LINDDUN uses similar DFD-based ideas from STRIDE, but its main focus was privacy and not legal jurisdiction. Because of that, there is no clear place in these frameworks to represent questions like "which country's courts can force access to this data". This is not the same as threats like spoofing or linkability. Sovereignty depends more on the legal and organisational relation between a customer and a service provider. The difference between what STRIDE/LINDDUN can describe and what a sovereignty threat means could be one reason why sovereignty was not added to these frameworks yet, instead of the reason being that this topic was completely ignored.

The third point is related to privacy support. It exists, but mostly as a separate area and it is not really included in the common security threat modeling tools. LINDDUN is already known in academic research and the KU Leuven LLM hybrid shows that parts of it can be automated while still keeping good accuracy. However, none of the four main tools discussed in Section 3.1 include something similar to a LINDDUN analysis. In real use, privacy threat modeling and security threat modeling are still treated as different areas, even if they often use the same type of system diagrams.

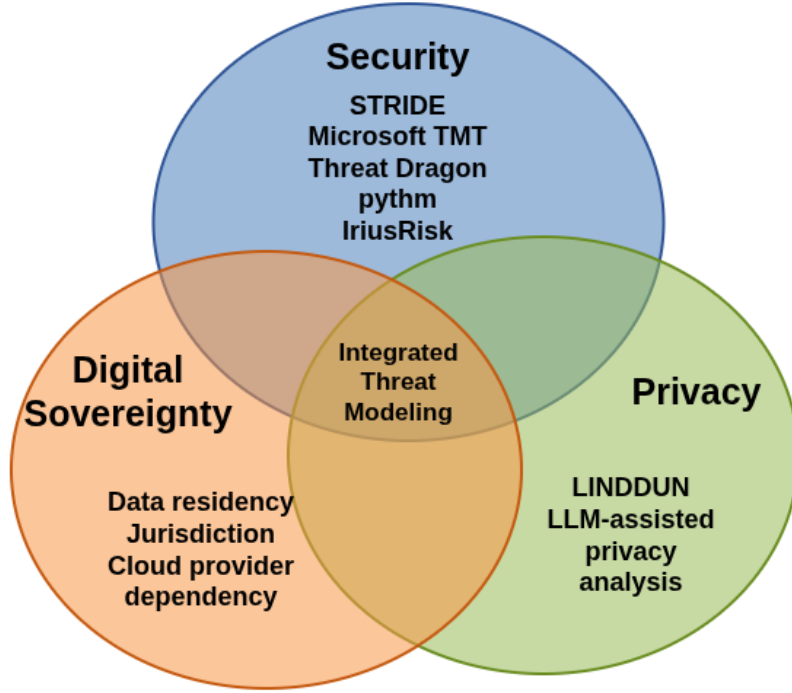


Figure 3.2: Overlap between security, privacy, and digital sovereignty in cloud threat modeling. Existing tools cover security well and privacy partially, but no tool addresses sovereignty, leaving the intersection of all three areas largely unaddressed. This gap is the focus of the combined taxonomy proposed in Chapter 6.

Lifecycle completeness, privacy integration, and sovereignty coverage have different problems behind them. Lifecycle completeness is often limited because of funding and practical priorities, privacy is separated between different research areas, and sovereignty is usually missing because existing taxonomies do not consider it as an important dimension. These gaps are not impossible to solve from a technical point of view. However, the reason they still exist is related to how current approaches are designed and used. Therefore, a possible contribution is not to create another STRIDE engine, because there are already different solutions available, but to extend the taxonomy with a dimension that was not included before and also show that lifecycle completeness can be addressed without depending on a commercial product.

3.3.1 Positioning of This Thesis

This thesis tries to address these gaps by answering the five research questions introduced in Chapter 1. RQ1–RQ3 and RQ5 are mainly answered with the comparative analysis presented in Chapter 5. In this analysis, sovereignty is also used as an evaluation dimension, which was not found in the published comparisons reviewed during this work. RQ4, which focuses on how sovereignty threats can be considered together with security and privacy threats, is answered in Chapter 6. For this purpose, a combined taxonomy is developed by

integrating STRIDE, LINDDUN, and a new sovereignty category. The goal is to avoid analysing the same system multiple times with separate threat models. The Proof-of-Concept described in Chapter 7 is used to test if this combined taxonomy can be applied in a real implementation. It is implemented as an open and reproducible pipeline and not as a commercial platform. This shows that a lifecycle-complete approach can also be achieved without using IriusRisk licensing, at least for a PoC implementation.

3.4 Chapter Summary

This chapter reviewed four commonly used threat modeling tools and three groups of research approaches based on LLMs. From this review, several limitations were observed. Some of them are related to the lifecycle coverage of the tools, where the available features are not always based on the technical challenges but also on the resources and development effort behind them. Privacy aspects are considered in some approaches, but they are usually separated from the main security threat modeling tools. Also, sovereignty is still not clearly included in existing methods because frameworks like STRIDE and LINDDUN were not originally designed to handle this type of requirement. These limitations shape the chapters that follow: Chapter 4 sets out the methodology, Chapter 5 presents the comparison, Chapter 6 introduces the taxonomy, and Chapter 7 describes the PoC implementation.

Chapter 4

Methodology

This chapter explains how the research was done. It describes the overall research design, the lifecycle model that is used for the evaluation, the criteria for scoring the tools, how the tools were selected, and also the main ideas behind the Proof-of-Concept.

4.1 Research Design

The thesis contains three main parts. The first part is a literature review about existing threat modeling tools, frameworks, and approaches based on LLMs (Chapter 3). The second part is a comparison of the selected tools where they are evaluated using a defined rubric (Chapter 5). The third part is a Proof-of-Concept implementation of the lifecycle-oriented approach presented in this work (Chapter 7).

For a bachelor thesis, it is important to include the theoretical part but also something practical that is developed. The literature review and the comparison are used to answer the research questions from Chapter 1. The PoC is used to see if the results from the comparison are also reasonable when applying them in an actual implementation.

4.2 Threat Lifecycle Model

Threat modeling here is not treated as a one-time activity. A model created once and never touched again goes stale fast ; new services get added, the architecture shifts, legal requirements change. So instead of a single pass, this thesis works with a cycle of four stages.

The four stages are based on NIST SP 800-30, the OWASP threat modeling process, and also the lifecycle guidance from the CSA Cloud Controls Matrix [14], [25], [33].

Table 4.1: Threat lifecycle stages used as the evaluation framework.

Stage	Description	Key Activity
1. Identification	Discover all relevant threats	DFD analysis, LLM generation
2. Categorisation	Classify threats by type	Apply taxonomy
3. Prioritisation	Rank threats by risk score	Likelihood $\times$ Impact
4. Mitigation	Define countermeasures and document findings	Map to control frameworks, generate

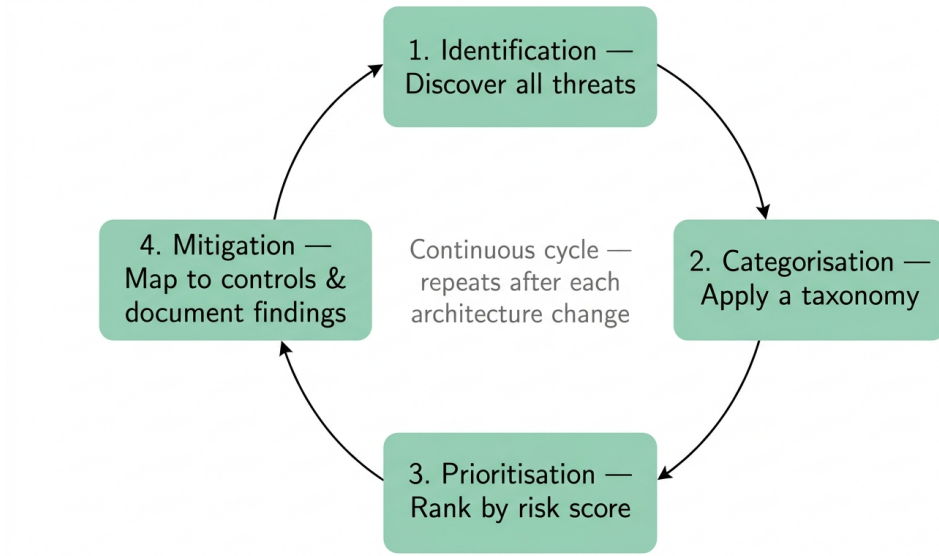


Figure 4.1: The four-stage threat lifecycle model, drawn as a circle rather than a straight line since the process starts over whenever the architecture changes in a meaningful way.

4.3 Evaluation Criteria

The comparison of the tools and frameworks is based on four dimensions. These dimensions were decided before looking at the individual tools, based on the gaps discussed in Chapter 3. This was done so the comparison criteria were already fixed and not changed later because one tool performed better in a specific aspect.

Table 4.2: Evaluation criteria for the comparative analysis.

Dimension	Description	Why It Matters
Lifecycle Coverage	Covers identification, categorisation, prioritisation, and mitigation	Directly required by RQ2
Cloud Applicability	Supports cloud-native architectures (K8s, IaaS/PaaS/SaaS)	Primary environment of this thesis
Privacy Support	Supports LINDDUN or equivalent privacy analysis	GDPR requires privacy by design
Sovereignty Relevance	Addresses jurisdiction, residency, foreign access	Key gap identified in related work

These four axes are taken directly from the proposal. Other properties, for example the automation level, LLM integration, and the licensing model, are not included as separate columns in the matrix. Instead, they are discussed in each tool description in Chapter 5 when they are useful to explain why a tool gets a specific score.

The proposal directly defines four comparison axes: lifecycle coverage, cloud applicability, privacy support, and sovereignty relevance. These correspond to the Lifecycle, Cloud Support, Privacy Coverage, and Sovereignty rows shown above. During the comparison, three more properties also appeared many times. One of them was automation level and LLM integration. This was already used informally as a column in Table 3.1 before writing this section, because several tools had the same score in the original four criteria and these features often showed the difference. Another one was reporting. A mitigation stage that does not produce any report or other output for users is not really complete in real use, even if reporting is not defined as a separate stage in Section 4.2. The last one was open-source licensing, because cost was also one reason why some tools have better lifecycle coverage than others, as discussed later in Chapter 5. These three properties are explained in Chapter 5, but they are not included as separate columns in the comparison matrix.

Each criterion uses a simple three-level scale: **Yes** (fully supported), **Partial** (limited or indirect support), and **No** (not supported). When there was a documented specification or a clear number available, it was used instead of only personal judgement. Chapter 5 explains how every score was assigned.

4.4 Tool Selection Criteria

Covering every tool on the market was not realistic ; the comparison would get too broad to be useful. Some filters were applied to narrow the selection down:

- Publicly available ; open source, or at least documented well enough so it can be read and understood.
- Still maintained, or if not, it should be used or cited in papers published after 2018.
- Supports at least one of these topics: cloud environments, privacy threat modeling, or automatic threat generation.
- The final group should contain one traditional DFD-based tool, one code-based tool, and one LLM-based approach. This was done to avoid comparing only one type of tool.

After applying these conditions, five tools remained for the comparison: Microsoft Threat Modeling Tool, OWASP Threat Dragon, IriusRisk, pytm, and LLM-based threat generation, which represents this category.

4.5 Proof-of-Concept Design Principles

The PoC was developed to show that it is possible to build a complete workflow which includes security, privacy and sovereignty in one solution, and not only discuss these ideas in theory. The implementation was mainly influenced by five design decisions.

Input is a **plain text description** of the cloud environment, not a formal diagram. This makes it easier to use because the user does not need to create a proper DFD before. The LLM can work directly from the written description, so the analyst only needs to describe the cloud environment in text.

It uses the **combined taxonomy** from Chapter 6. In this way, STRIDE, LINDDUN and sovereignty threats are classified with one common scheme, so it is not necessary to do three different analysis passes.

It aims to have **lifecycle completeness** ; all four stages are implemented, and the output from one stage is used as the input for the next stage.

Risk scoring and mitigation mapping are **implemented with rule-based Python code**, not generated by the LLM. We decided to do it this way because the scores and mitigation results are always produced from the same rules. So it is easier to reproduce the output and also check which rule caused the result, instead of depending on what the model answers in one execution.

The LLM component runs **entirely locally** by using Ollama, so no data is sent to an external server. If a company's architecture description was sent to a US-hosted API, it could create the same jurisdiction problems which this thesis tries to avoid. Because of this, using a local model was the only choice that was matching the main idea of this thesis.

4.6 Limitations of the Methodology

- The comparison mostly uses documentation and published papers instead of testing every tool directly. Because of that, some documented features maybe are different from what happens in real use.
- The Yes/Partial/No rating is subjective, so another person checking the same tools could probably give some different scores.
- The PoC was tested only with simplified system descriptions. Before making stronger conclusions, it should also be checked in a more complex production environment.
- LLM output is not deterministic, so using the same input again can still generate a little different list of threats.

4.7 Chapter Summary

This chapter explained the research design and the four-stage lifecycle model. It also described the four comparison criteria, how the tools were selected, and the five principles used for the PoC. The following chapters build directly on these parts.

Chapter 5

Comparative Analysis

Chapter 4 described four lifecycle stages and also four comparison criteria from the proposal. These criteria are lifecycle coverage, cloud applicability, privacy support, and sovereignty relevance. In this chapter, these criteria are used to compare the five selected tools and approaches from Section 4.4.

A quick note on what kind of comparison this actually is, because it is not a benchmark in the strict sense. Installing all five tools and running the same system through each one would be a thesis on its own ; and IriusRisk and the LLM-based approach are not even used the same way to begin with, which makes a direct test difficult anyway. So the comparison leans on documentation, source code where I could get to it, and published case studies, scored against the rubric from Chapter 4. That is not a substitute for hands-on testing, but I think it is a defensible way to do this given the scope. Keep that limitation in mind for every result in the table below.

5.1 Comparative Analysis Matrix

Table 5.1 presents the complete comparison matrix. In the table, Y means the criterion is supported, P means it is only partly or indirectly supported, and N means there is no support for this criterion.

Table 5.1: Full comparative analysis matrix. Y = Yes, P = Partial, N = No.

Criterion	MS TMT	Th. Dragon	IriusRisk	pytm	LLM-based
Cloud Applicability	P	P	Y	Y	Y
Privacy Support	N	N	P	N	P
Sovereignty Relevance	N	N	N	N	N
Lifecycle: Identification	Y	Y	Y	Y	Y
Lifecycle: Categorisation	Y	P	Y	Y	P
Lifecycle: Prioritisation	P	N	Y	P	P
Lifecycle: Mitigation	P	N	Y	N	P

Before looking at every dimension in the table, two rows should be mentioned first. Sovereignty Awareness is the most obvious result because all tools have **N** in this row. Things like jurisdiction, data residency and foreign government access are not discussed in the documentation, whitepapers or public issue trackers of any of the five tools. Section 5.2.2 explains that this is probably a more general limitation and not only because some information was missed. Automation is different, so it is explained in Section 5.2.4 instead of being evaluated in the table. In Table 3.1, pytm has the highest automation level (High), while MS TMT and Threat Dragon are the lowest (Low). IriusRisk has Medium automation, but it covers more parts of the threat modeling lifecycle than the other tools. The LLM-based approach is also not included in the scoring table. Still, Section 5.2.4 discusses that in real use it could automate more than what the table alone may suggest. In these tools, better automation does not always mean better lifecycle coverage. From this small comparison, they seem to have some trade-off, and this is discussed again later in Section 5.4.

5.2 Analysis by Dimension

5.2.1 Cloud Support

Every tool gets at least P for cloud support, but this does not mean they support cloud in the same way. Microsoft TMT and OWASP Threat Dragon can model cloud systems, but basically they work the same as for other systems. They let users draw boxes and arrows. If a box is named "S3 Bucket" instead of "File Server", it does not really change how the tool works because both are generic DFD editors and they do not have built-in knowledge about cloud services. For example, a team can draw a Kubernetes cluster in both tools, but the tools do not know that a Kubernetes cluster has things like the API server, etcd, or pod-to-pod traffic which can introduce different attack surfaces compared to a normal file server.

IriusRisk and pytm provide more support because they already include AWS and Azure components in their pattern libraries. If a user selects an S3 bucket, the generated threats are related to S3 and not only general threats for a data store. LLM-based approaches are different because they do not require drawing a diagram. They read a text description of the architecture. This can help because in many projects the architecture changes often and diagrams are not updated regularly. Keeping a text description updated is usually easier. Still, this mainly helps with maintaining the documentation. It does not automatically mean the model understands cloud attack surfaces better than a rule-based library. This depends a lot on the prompt, as discussed later in Section 5.2.4.

There are also things that none of the five tools cover. For example, serverless cold-start data exposure, API-gateway rate-limiting bypass, and known container-escape techniques are missing. This is because their control catalogues, including the CSA CCM [25], were not designed around these specific attack techniques. Because of that, "cloud support" in this comparison mostly means that a tool can recognize cloud infrastructure and distinguish something like an S3 bucket from a file server. It does not mean the tool has good coverage of cloud attack techniques. So the Y and P scores in the table can give a wider impression than what is actually supported.

5.2.2 Privacy and Sovereignty

The sovereignty result is not complicated: none of the five tools touch it, even partially. IriusRisk came closest to earning partial credit at first glance, since it has a GDPR checklist. But that checklist is about consent and data-subject rights ; can a user get their data deleted, that kind of thing. It has nothing to do with sovereignty, which is really about which country's courts can force access to the data regardless of what the user agreed to. A tool can tick every GDPR checklist box and still happily route personal data through a jurisdiction with zero sovereignty protection. Section 6.3.1 shows exactly this situation.

Privacy is covered a little better, but overall it is still limited in these tools. The LINDDUN-LLM hybrid from KU Leuven is the strongest privacy-related approach in this review because it performs a real LINDDUN analysis. It checks the data flow for linkability, identifiability, and the other LINDDUN categories one by one. So it is not just about looking for PII fields that may contain personal data. IriusRisk can help for GDPR audits, but this is not the same as LINDDUN analysis. It mainly checks whether the required items are completed, but it does not analyse if one specific data flow could create a linkability problem between different services. Microsoft TMT, Threat Dragon, and pytm also do not include privacy analysis. These three tools were developed mainly for STRIDE threat modelling, and STRIDE itself does not have concepts for privacy threats.

5.2.3 Lifecycle Completeness

IriusRisk is the only tool that covers the complete lifecycle, starting from threat identification until producing a report that an auditor can use directly. The other tools mainly focus on identifying threats, but after this stage the support becomes weaker. Threat Dragon is the clearest example because after the categorisation step it almost does not provide more support. This result was also seen when comparing four different tools.

Identification is the main part of threat modeling methods. For example, STRIDE is mainly used by checking threats on a DFD. So, developers usually spend more effort on this stage first. Reporting and mitigation tracking seem to get less attention. They are also connected more with how an organisation works than with the threat generation itself. Maybe this is one reason why many open-source projects made by volunteers do not finish these features because they have limited time. Section 5.2.4 and Chapter 3 discuss this point again.

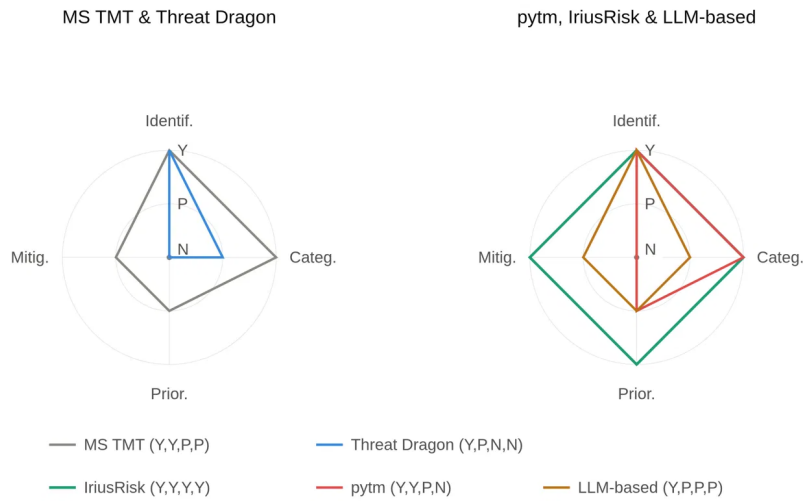


Figure 5.1: Lifecycle completeness across the five evaluated tools. Each axis is one lifecycle stage, scored N/P/Y as in Table 5.1. IriusRisk is the only tool that reaches the outer edge on every axis.

If a tool only stops at categorisation, a team loses time elsewhere. In many cases, they still need to do the next work manually or use different tools. For example, risk scoring can be kept in a spreadsheet, mitigation tasks in a Jira board, and the final audit report written in a Word document. These things are usually not connected with the original threat model. Therefore, information can be missed when moving between tools, and some threats may not be tracked correctly. A tool that supports the whole lifecycle is supposed to reduce this kind of problem.

5.2.4 Automation and LLM Integration

It should also be mentioned that, outside the formal rubric, both pytm and the LLM-based approach are quite automated, but not for the same reason. In pytm, the system description is already written as code, so the model can be generated automatically. For the LLM approach, it does not need a formal modelling step, but then the results are less deterministic. This point was not included in the scoring because the four evaluation axes do not consider automation. Still, it gives some useful background for the design choice in Chapter 7, where the scoring and mitigation stay deterministic no matter which threat identification method is used.

5.3 Summary of Findings

1. No reviewed tool covers the complete lifecycle and also supports privacy and sovereignty together. The closest examples are IriusRisk, which has very good lifecycle support but no sovereignty support, and the LINDDUN-LLM hybrid, which focuses more on privacy but only covers a small part of the lifecycle. This is similar to the trade-off explained in Section 5.2.3, where improving one side usually means less coverage in another area.
2. Sovereignty is the biggest missing part in the comparison. Every tool gets an N for this criterion. As discussed in Section 5.2.2, this seems to be a more general problem and not only because vendors did not implement this feature.
3. Privacy methods already exist in research, but they are not common in tools that are used in a real setting. LINDDUN is well known in academic work, however none of the reviewed tools actually integrate it. Most of them use simpler compliance checklists instead.
4. IriusRisk has the best lifecycle coverage of everything evaluated here, full stop. But it is commercial, which rules it out for a reproducible academic PoC, and the license cost puts it out of reach for a lot of smaller teams too. The completeness and the commercial model are two sides of the same coin.
5. LLM-based approaches seem to be a promising direction because they could improve both privacy support and automation. At the moment the automation score is still limited, but this may not show the full potential. Better engineering and additional supporting components could improve the practical performance.

The sovereignty gap explained in Chapter 6 is the main reason for the taxonomy presented there. Also, the automation/lifecycle gap discussed in Section 5.2.4 was used as a basis when designing the PoC in Chapter 7.

5.4 Chapter Summary

Five tools, four criteria, one of them split into four lifecycle stages ; and the pattern across all of it is fairly blunt. Sovereignty support: missing everywhere. Privacy: better, but still thin. Lifecycle coverage tracks commercial investment more than it tracks technical difficulty. IriusRisk comes out on top overall, but it is commercial and still has nothing for sovereignty. The taxonomy and PoC that follow build directly on this gap.

Chapter 6

Privacy and Sovereignty in Cloud Threat Modeling

Chapter 5 showed two important gaps. Existing tools only cover privacy partly, while sovereignty is not considered. In this chapter, RQ4 is answered by making a taxonomy that combines STRIDE, LINDDUN and an additional sovereignty category in one classification. The taxonomy was selected because it is easier to organize the different sovereignty threats into specific sub-categories instead of discussing them only in a general way. It also gives the structure which is used later for the PoC in Chapter 7.

6.1 Integrating Privacy Threats into Cloud Threat Models

6.1.1 The Distinction Between Security and Privacy

Security and privacy are often discussed together because they use many similar terms, but they are not the same. Security threat modeling is mainly about what an attacker may do to attack or misuse a system. Privacy threat modeling looks more at what the system itself can do with users' data. This can happen even if there is no attacker, because the system is working in the normal way it was designed.

Take a company which has good technical security measures, for example it encrypts the database, uses MFA for the systems, applies security patches regularly and never had a data breach. From a cybersecurity point of view, this company can be considered secure. However, if this company stores customer purchase history for ten years after the user closed the account only because it may be useful later, it can still create a privacy problem. This issue exists even if there was no hacking incident or data leakage. GDPR Article 25 explains this approach by requiring privacy protection to be considered during the design of systems and processes, instead of trying to fix privacy problems after they appear [2].

Cloud systems make this problem more difficult because personal data can move through many different components, and people often do not see all of these steps. For example, a signup form can store information in a main database, then this data can be copied to another database in a different region. It can also appear in API gateway logs, monitoring systems, or be sent to external marketing services. Because of these different points, every part of the process can introduce a possible LINDDUN threat. The difficulty is also that these parts are often managed by different teams, and they may see them mainly as technical infrastructure instead of as parts where personal data processing happens [18].

6.1.2 Applying LINDDUN in Cloud Environments

LINDDUN uses the same Data Flow Diagrams as STRIDE, which means that the same DFD can also be used for both approaches. However, there is a practical difference when creating these diagrams. Security-focused DFDs often only show the application boundary and the main components around it. In cloud systems, many privacy related data flows are happening in lower layers, for example cross-region data replication, cloud provider telemetry, and data sharing with third-party sub-processors. These parts are usually missing from the diagrams because the people creating them are more focused on the system infrastructure and security issues, not always on where personal data is moving.

All seven categories of LINDDUN can appear in cloud environments, but in actual use some categories are seen more frequently than the others.

- **Linkability:** activities of one user from different services can become connected, even when this was not planned. This can happen for example because the same session token is used, a similar user ID exists, or a logging system collects information from different services together.
- **Identifiability:** personal information can appear in places where it should not be, especially in logs or monitoring tools. These systems are normally used for debugging problems, but they can have less strict access permissions and keep data for a longer time compared to the main database.
- **Unawareness:** the user often does not know which other companies or sub-processors receive the data, and also does not know in which countries this data is processed.
- **Non-compliance:** data can be stored for more time than the privacy policy says, or it can be processed in a country where there is no valid legal basis. Another problem is using the data for a reason which the user did not accept before.
- **Non-repudiation:** a user can say that they did not perform an action, for example deleting data or removing consent, and the system may not have enough evidence to

show what really happened. This is important for GDPR requests because an audit history is needed in these cases.

- **Detectability:** it can be possible to find out that a user’s data exists without directly seeing the data. For example, different response times or error messages can show if a specific account is registered in a system.
- **Disclosure of Information:** personal information is shown to a person or system which should not have access to it. It differs from Identifiability: the information does not always need to be connected to a specific person to create a privacy problem.

All seven categories are included in the taxonomy below, because also the three categories which are more important for anonymous-voting or whistle-blower systems, namely Non-repudiation, Detectability, and Disclosure of Information, were still relevant during the evaluation. These categories appeared when the PoC from Chapter 7 was tested with a real cloud description.

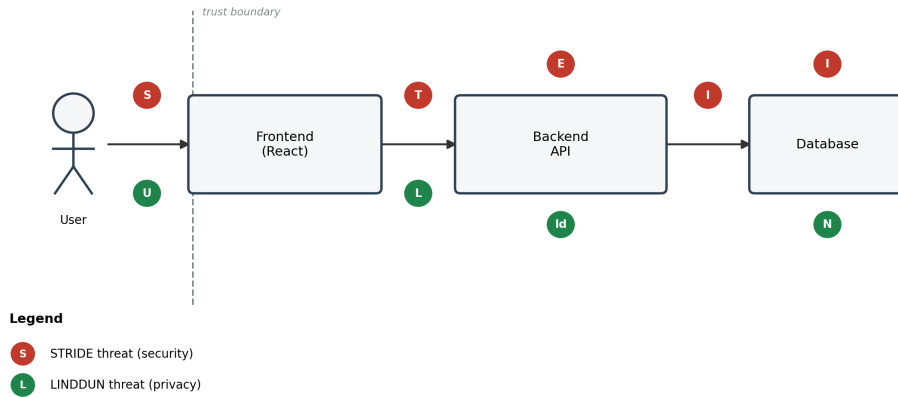


Figure 6.1: STRIDE and LINDDUN applied to the same simplified Data Flow Diagram. Red markers show STRIDE (security) threats, green markers show LINDDUN (privacy) threats. The flow between frontend and backend, for example, carries both a Tampering threat and a Linkability threat.

6.2 Sovereignty Threat Modeling

6.2.1 Defining Sovereignty Threats

No existing framework was found that includes sovereignty aspects, therefore this part of the taxonomy was developed by using legal and policy sources. It was not based on an already existing methodology such as STRIDE or LINDDUN, because these frameworks do not consider this topic.

For this thesis, a sovereignty threat means a situation where an organisation can lose control over its own data or infrastructure because of external factors which are not directly part of the technical system. This can happen, for example, because of a foreign court decision, a subpoena, changes in another country's regulations, or because an organisation depends too much on a proprietary platform. In the last case, changing to another provider may become very difficult or not realistic anymore.

Four patterns were found several times during the research, especially when looking at European organisations:

- **Jurisdictional exposure:** a provider which has its headquarters in the US may be required to provide data because of the CLOUD Act, even if the data itself is stored in another country or region [3].
- **Foreign access:** this is connected with jurisdictional exposure, but it is a little different. A government can get access to data from a provider and the provider may not be allowed to tell the customer that this access happened.
- **Vendor lock-in:** this is not a legal problem directly, but it can create problems for cloud sovereignty. When a company uses many proprietary APIs or specific managed services, moving to another provider can become very difficult and costly.
- **Data residency violation:** this happens when data is stored outside of the required region. It can happen because of replication or backup settings which were not checked correctly, and not always because someone intentionally moved the data.

6.3 Combined Threat Taxonomy

STRIDE is mainly used for security, while LINDDUN is focused on privacy, but both of them do not include sovereignty aspects. The taxonomy presented below puts these three areas together in one framework and applies them to the system in the same analysis step, instead of doing three different analyses separately.

Table 6.1: Combined security-privacy-sovereignty threat taxonomy for cloud environments.

Category	Sub-category	Example Threat	Origin	Standard
Security	Spoofing	Impersonate cloud service identity	STRIDE	CIS, NIST
Security	Tampering	Modify data between microservices	STRIDE	ISO 27001
Security	Repudiation	Disable cloud audit logs	STRIDE	CSA CCM
Security	Info. Disclosure	Misconfigured S3 bucket exposes data	STRIDE	CIS, NIST
Security	Denial of Service	Overwhelm API gateway	STRIDE	NIST
Security	Elev. of Privilege	Exploit overpermissive IAM role	STRIDE	CIS, NIST
Privacy	Linkability	Link user actions across services	LINDDUNGDP	Art. 25
Privacy	Identifiability	PII present in logs unnecessarily	LINDDUNGDP	Art. 5
Privacy	Unawareness	User unaware of sub-processor locations	LINDDUNGDP	Art. 13
Privacy	Non-compliance	Data retained beyond GDPR limit	LINDDUNGDP	Art. 5(e)
Privacy	Non-repudiation	User denies deleting a record	LINDDUNGDP	Art. 17
Privacy	Detectability	Timing/error leak reveals account exists	LINDDUNGDP	Art. 25
Privacy	Disclosure of Info.	Personal data exposed to wrong party	LINDDUNGDP	Art. 32
Sovereignty	Jurisdictional Exp.	US provider subject to CLOUD Act	Novel	GDPR Ch. V
Sovereignty	Foreign Access	Provider compelled by foreign government	Novel	GDPR Ch. V
Sovereignty	Vendor Lock-in	Proprietary APIs prevent provider switch	Novel	Gaia-X
Sovereignty	Data Residency Viol.	Data replicated outside EU on failover	Novel	GDPR Art. 44

The security row is still STRIDE and it was not changed. The privacy row includes the seven LINDDUN categories that were selected in Section 6.1.2. The sovereignty row is the part that I developed by myself. The four sub-categories are based on information from GDPR Chapter V, the CLOUD Act, and the Gaia-X Trust Framework. I selected them because similar problems related to control and data access can be found in these three sources.

6.3.1 Worked Example: Applying the Taxonomy to a Single Data Flow

Here is a simple example showing how the taxonomy can be used. For example, a signup form in a React frontend collects the user's name and email. Then this information is sent to a backend service, where it is stored in a database. After this, the backend can use an email provider from the US for sending a welcome email to the new user.

Running the taxonomy on this flow shows that there is a threat identified in every category.

- **Security (Tampering):** if the connection between the frontend and backend is not authenticated, an attacker could change the signup data while it is being sent. For example, the email address in the request could be replaced with another one.
- **Privacy (Identifiability):** when the email address is saved in the backend logs, which can happen by default in many web frameworks, it creates another copy of this personal data. These logs can have different access rules and they can also be stored for a longer time compared to the user database.
- **Sovereignty (Jurisdictional Exposure):** the welcome email service is provided by a US company, so the user's name and email address are leaving the EU backend. This means the data can also be affected by US laws and legal requests, even if the main system is hosted in the EU.

STRIDE alone would not be enough to identify these three threats. Even when STRIDE is combined with LINDDUN, the third threat can still be missed. This shows why the taxonomy is needed, because using only one view of threats is not always enough. By including security, privacy, and sovereignty together, more types of threats can be considered during the analysis, instead of only putting labels on threats after they are already identified.

6.4 Chapter Summary

This chapter extended LINDDUN for cloud-related data flows and added a sovereignty category based on GDPR Chapter V, the CLOUD Act, and Gaia-X. These parts were

combined into a taxonomy with seventeen sub-categories. The example in this chapter shows that the taxonomy is not only for assigning labels to threats. In one realistic data flow, threats from all three categories can appear at the same time, which would not be found together by a single-framework tool discussed in Chapter 5. The developed taxonomy is later used as the basis for the PoC presented in Chapter 7.

Chapter 7

Proof-of-Concept Implementation

The comparison and the taxonomy so far are mostly theory ; they describe what should be possible on paper. This chapter is where I actually build something to check if that holds up. The PoC is not meant to be production software, nothing close to it. What I wanted to show is that the four steps ; identification, categorisation, prioritisation, mitigation ; can actually run together as one pipeline, and that this pipeline covers security, privacy, and sovereignty at the same time instead of three separate passes.

7.1 Requirements

These are based on the four PoC steps from the proposal, the research goals described in Chapter 1, and also the gaps that were identified in Chapter 5.

Table 7.1: PoC functional and non-functional requirements.

ID	Requirement	Type	Priority
R01	Accept a textual description of a cloud system as input	Functional	Must
R02	Identify threats using the combined taxonomy (Sec./Priv./Sov.)	Functional	Must
R03	Categorise each threat by taxonomy category and sub-category	Functional	Must
R04	Assign a risk score based on likelihood and impact	Functional	Must
R05	Suggest mitigations from NIST, CIS, GDPR, or Gaia-X	Functional	Must
R06	Produce a structured threat report as output	Functional	Must
R07	Support cloud-native descriptions (Kubernetes, microservices)	Functional	Should
R08	Use LLM assistance for threat generation	Functional	Should
R09	Be reproducible and fully documented	Non-functional	Must
R10	Run without internet or cloud infrastructure (local execution)	Non-functional	Must

R01–R05 correspond to the four steps of the proposal. R06–R10 are more about the implementation choices, because these four steps also need to work as one reproducible pipeline, not only as a script that is used one time.

7.2 Technology Stack

The PoC is implemented as one Python script, so it is possible to read the complete pipeline from the beginning to the end in a single file. The stack:

- **Ollama:** a local server for running open-source LLMs on the local machine. It does not need an external API or internet connection and is available on `localhost:11434` [34].
- **Llama 3 (Meta):** the language model used to identify threats. The model runs locally on the computer, so the system description is not sent outside the machine [35].
- **requests:** used for sending HTTP requests to Ollama.

- **re:** used to extract the JSON array from the model output because sometimes the model adds an extra sentence before or after the JSON, even if the prompt asks only for JSON.
- **json:** used to convert the JSON array into Python objects.

A local model was really the only option that made sense here. Sending a company’s cloud architecture description to a US-hosted API would have created exactly the jurisdiction problem this thesis is arguing against ; which would have been a bit embarrassing. So everything runs locally through Ollama, which lines up with the sovereignty argument from Chapter 6. The downside: my machine only has a CPU, and one run took several minutes. More on that in Section 7.6.

7.3 System Architecture

The PoC follows the four steps from the proposal. One LLM call is used for both identification and categorisation (Stages 1–2). After this step, the remaining processing is implemented with deterministic Python code.

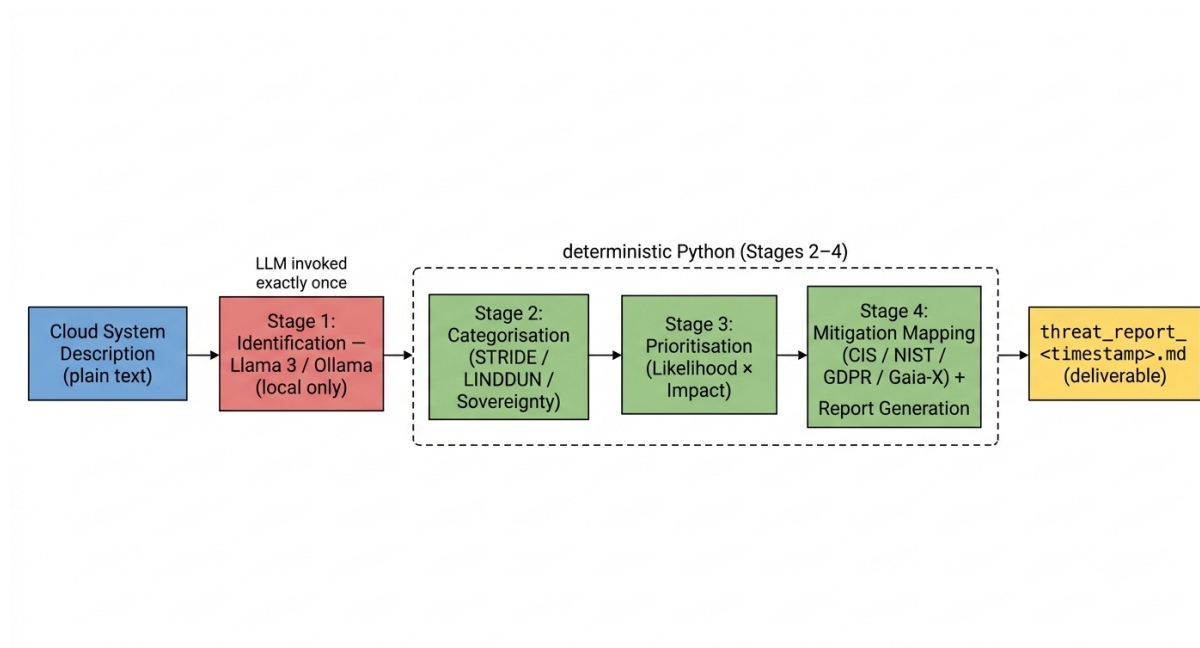


Figure 7.1: PoC pipeline architecture. A single LLM call performs both Stage 1 (Identification) and Stage 2 (Categorisation), since the model returns each threat’s category and sub-category directly. Stages 3–4 are deterministic Python code, dashed to distinguish them from the single non-deterministic step.

1. **Input:** the cloud system description written in normal English text.

2. **Prompt construction:** `build_prompt()` puts this description into a prompt. It also includes all seventeen taxonomy sub-categories and tells the model to return only JSON, without explanation or markdown.
3. **Identification (Stage 1):** Ollama generates the response as raw text. In most cases it is only the JSON array, but sometimes there is one extra sentence before or after it.
4. **Parsing:** `parse_threats()` finds the [...] part using a regex and then loads it with `json`. If the parsing does not work, the script prints a warning together with part of the raw output instead of stopping without any message. Llama 3 on CPU sometimes still starts with "Here is the JSON:" even if the prompt says not to do this.
5. **Categorisation (Stage 2):** for every detected threat, the model gives a **category** and **subcategory**. It should only use the categories which are already listed in the prompt.
6. **Prioritisation (Stage 3):** `score_threat()` checks the sub-category in the fixed `SCORING_RULES` dictionary. This dictionary has manually assigned likelihood and impact values from 1 to 3. The risk score is calculated as $\text{round}(\text{Likelihood} \times \text{Impact}/9) \times 10$, with the maximum value of 10. Threats with score 7 or more are marked as HIGH.
7. **Mitigation (Stage 4):** `add_mitigation()` adds one predefined mitigation for each sub-category. The mitigation points to a related control from CIS, NIST or GDPR. For example, Elevation of Privilege is linked with CIS AWS 1.16 and CIS Kubernetes 5.1. After this, the threats are ordered by the risk score and shown in a summary table. So, there is no need to make another reporting stage to satisfy R06.

I could have let the LLM handle scoring and mitigation too ; it would have made the implementation simpler. But I ran into a problem pretty quickly: run the same input twice and the model does not always give the same priority to the same threat. That is not something I wanted in a report people are supposed to make decisions from. So scoring and mitigation stayed as fixed, rule-based lookups instead ; same threat, same score, same mitigation, every time.

7.4 LLM Prompting Strategy

The prompt contains the three top-level categories and also the sub-categories under each of them from the taxonomy in Chapter 6. These sub-categories are copied verbatim. For every threat that the model identifies, it returns six fields only: **name**, **category**,

subcategory, component, description, and justification. No other output should be generated.

Two choices are mainly used to reduce hallucination. First, the output must be in structured JSON and it only can use predefined sub-categories. Therefore, the model cannot easily create new categories which are not in the taxonomy. If it still gives something that cannot be matched, the mitigation lookup uses a generic NIST reference. This way the wrong result is still visible and not looking correct by mistake.

The second choice is setting the temperature to 0.2. It is still not completely deterministic, but usually it makes the answers more stable. Also it reduces some generic or boilerplate threats that are not really related with the given system description.

This is still not completely correct. As Section 7.5.3 shows, the closed list can stop the model from creating a new sub-category. But it still can assign a sub-category with the correct name to a wrong top-level category.

7.5 Example Use Case

7.5.1 System Under Analysis

The PoC was tested using the following description:

The application is deployed on AWS as a web application. The frontend uses React and it is served with AWS CloudFront. On the backend, there are three microservices which run on Kubernetes (AWS EKS). For user login and authentication, AWS Cognito is used. User information like names and email addresses are stored in a PostgreSQL database on AWS RDS in eu-west-1 (Frankfurt). The system also sends email notifications using Mailgun. Mailgun is a company based in the US. For CI/CD, GitHub Actions is used. The developers also have administrator access to the Kubernetes cluster.

I picked this setup on purpose: an EU-hosted database, a US-based email provider, and broad Kubernetes admin access thrown in for good measure. The goal was simple ; see if the pipeline actually catches threats from all three taxonomy categories, not just the security ones.

7.5.2 Results

The run found 10 threats in total: 3 Security, 4 Privacy and 3 Sovereignty. Among them, 8 were scored as HIGH, 1 as MEDIUM and 1 as LOW. The complete results are shown in Table 7.2. The table is ordered by the risk score and the values are the same as produced by the tool.

Table 7.2: PoC output threats for the example AWS system, sorted by risk score.

Threat	Category	Sub-category	Score
Kubernetes Elevation of Privilege	Security	Elevation of Privilege	10/10 ; HIGH
RDS PostgreSQL Information Disclosure	Security	Information Disclosure	10/10 ; HIGH
Jurisdictional Exposure EU-west-1	Sovereignty	Jurisdictional Exposure	10/10 ; HIGH
CloudFront Spoofing	Security	Spoofing	7/10 ; HIGH
Mailgun Data Residency Violation	Sovereignty	Data Residency Violation	7/10 ; HIGH
GitHub Actions Non-compliance*	Sovereignty	Non-compliance	7/10 ; HIGH
RDS PostgreSQL Identifiability	Privacy	Identifiability	7/10 ; HIGH
CloudFront Disclosure of Information	Privacy	Disclosure of Information	7/10 ; HIGH
AWS Cognito Linkability	Privacy	Linkability	4/10 ; MEDIUM
Kubernetes Detectability	Privacy	Detectability	1/10 ; LOW

*See Section 7.5.3: this item is filed under Sovereignty, but "Non-compliance" is a LINDDUN privacy sub-category ; a misclassification produced by the model, not by the scoring or mitigation logic.

```
sova@edr23346:~/Desktop/Study/Bachelor_Arbeit/PoC$ python3 threat_analyzer.py

=====
CLOUD THREAT ANALYZER – PoC | Bachelor Thesis | Bechir Sayadi | TU Dresden
=====

Running locally via Ollama – no data leaves this machine.

[Stage 1] Sending system description to local LLM (Ollama)...
[Stage 1] Parsing identified threats...
[Stage 2-4] Categorizing, scoring, and mapping mitigations for 10 threats...

=====
CLOUD THREAT MODELING REPORT – PoC | Bechir Sayadi | TU Dresden
Generated: 2026-07-24 10:27
=====

Total: 10 | Security: 3 Privacy: 4 Sovereignty: 3 | HIGH: 8 MED: 1 LOW: 1
-----
SCORE CAT      SUBCATEGORY      THREAT
-----
10/10 Security  Elevation of Privilege  Kubernetes Elevation of Privilege
10/10 Security  Information Disclosure  RDS PostgreSQL Information Disclosure
10/10 Sovereignty Jurisdictional Exposure  Jurisdictional Exposure EU-west-1
7/10 Security  Spoofing              CloudFront Spoofing
7/10 Sovereignty Data Residency ViolationMailgun Data Residency Violation
7/10 Sovereignty Non-compliance              GitHub Actions Non-compliance
... +4 more threats (see full Markdown report)
=====

Report saved: threat_report_20260724_102715.md
sova@edr23346:~/Desktop/Study/Bachelor_Arbeit/PoC$
```

Figure 7.2: PoC terminal output for the example AWS application, generated directly by `threat_analyzer.py`. The sovereignty threats (Jurisdictional Exposure, Data Residency Violation) are findings no evaluated tool from Chapter 5 would produce.

7.5.3 Key Finding

This is probably the most useful result to come out of the whole run. For the Jurisdictional Exposure item, the model picked up on the fact that the database sits in eu-west-1 and used that on its own to reason about jurisdiction risk. It also flagged Mailgun as a US company and connected that to data residency concerns ; without me ever mentioning "CLOUD Act" or "Schrems II" anywhere in the prompt, only the bare sub-category names. None of the five tools from Chapter 5 could have produced this, simply because none of them have a sovereignty category to reason from in the first place.

The same run also produced one misclassification, which I think is worth being honest about here. *GitHub Actions Non-compliance* was classified with `category: "Sovereignty"` and `subcategory: "Non-compliance"`. However, "Non-compliance" is one of the seven LINDDUN sub-categories used in this taxonomy (Section 6.1.2), and it belongs to the

Privacy category. The fixed list of sub-categories prevented the model from creating another label, but it still selected the correct sub-category with the wrong parent category. Since `add_mitigation()` uses the sub-category instead of the category, the system still assigned the correct GDPR data-retention mitigation. For this reason, the mistake only appears in the classification column and it does not continue to the mitigation result. This example shows why the scoring and mitigation are still implemented in deterministic code (Section 4.5). The model output should still be checked before the categorisation result is accepted.

7.6 Limitations of the PoC

- The model sometimes chooses the correct LINDDUN sub-category but puts it under the wrong main category. This happened for the *GitHub Actions Non-compliance* example discussed before. The closed taxonomy helps to avoid creating new categories, but it cannot stop this type of classification mistake. Because of that, the generated output should still be checked by a person before using it.
- The model also did not cover all LINDDUN sub-categories equally. In this test it found five from seven categories: Identifiability, Disclosure of Information, Linkability, Detectability and Non-compliance. However, Non-compliance was placed under Sovereignty (see Section 7.5.3), and Unawareness and Non-repudiation were not found. One possible reason is that the local CPU model is smaller than models such as GPT-4, so it may have more difficulty with privacy threats because they often need more context.
- The risk scores are fixed for every sub-category and they do not change depending on the analysed system. For example, an Elevation-of-Privilege threat in Kubernetes gets the same likelihood and impact values even if one cluster has only three administrators and another has thirty. This makes the results easier to reproduce, but the scores should be seen only as default values and not as a complete risk assessment.
- The PoC was evaluated only with one simplified system description similar to examples from textbooks. It was not tested with larger production systems that may include legacy software or integrations without documentation. Such descriptions could be more difficult for the model to understand correctly, but this work did not investigate that.
- The LLM output is also not always exactly the same between different executions, even when using a low temperature value of 0.2. This behaviour is already discussed

in Section 3.2.1. Because of that, the exact number of generated threats may be a little different if the experiment is repeated.

- The experiments were performed on CPU-only hardware, and one execution needed several minutes to finish. For more practical use, especially interactive use, it would probably need GPU acceleration or a smaller, quantised model because otherwise the pipeline is mainly suitable for batch execution.

7.7 Chapter Summary

This chapter implemented the four PoC steps from the proposal in one pipeline. A locally hosted LLM was used for identifying and categorising the threats in one call, while the prioritisation and mitigation mapping were done by deterministic Python code. When testing with the example system, the pipeline found threats from all three taxonomy categories. It also identified sovereignty threats, which were not found by the tools compared in Chapter 5.

During the execution, there was also one categorisation mistake from the model. This error was not removed because it shows one limitation of the approach. Because of this result, the scoring and mitigation parts were kept deterministic, but the categorisation from the LLM still should be checked before using the final result.

Chapter 8

Evaluation and Discussion

With the comparison, the taxonomy, and the PoC done, this chapter checks the work against what the proposal actually asked for, answers the five research questions from Chapter 1, and sets out what I would tell someone who wanted to use this for real.

8.1 PoC Requirements Evaluation

The proposal described the PoC around four steps: identification (Aufnahme), categorisation (Kategorisierung), prioritisation (Priorisierung), and mitigation (Mitigationsmaßnahmen). Table 7.1 in Chapter 7 broke these down into ten concrete requirements. Table 8.1 revisits that list now that the PoC has actually run.

Table 8.1: PoC requirements fulfilment assessment.

Req.	Status	Notes
R01	Met	Implemented and tested
R02	Met	All three taxonomy categories covered
R03	Met	Full taxonomy classification implemented
R04	Met	Likelihood $\times$ Impact scoring implemented
R05	Met	NIST/CIS/GDPR/Gaia-X lookup table implemented
R06	Met	Terminal report generated
R07	Partial	Tested on AWS/Kubernetes only; not tried on Azure, GCP, or a serverless-heavy architecture
R08	Met	Llama 3 via Ollama integrated and tested
R09	Met	Code and prompts fully documented
R10	Met	Runs fully locally with Ollama, no internet required

R07 is the only item which is marked as Partial instead of Met. The reason is that the taxonomy and prompt were designed in a general way, so they should not depend on a specific cloud provider. However, during the PoC testing I only used an AWS-based description because this was the example used in the thesis. I did not test if the same approach works with Azure or GCP terms. As a result, I marked it as Partial instead of assuming that it will also work correctly on other cloud providers without additional testing.

The four steps from the proposal are connected with the Stages 1–4 of the PoC pipeline (Chapter 7). Therefore, the PoC implementation includes the main parts that were described in the proposal.

8.2 Research Questions Revisited

Table 8.2: Research questions and corresponding answers from thesis findings.

Research Question	Answer	Evidence
RQ1: Which benchmarks are most relevant?	CIS and CSA CCM for cloud-specificity; NIST for breadth	Chapter 2, PoC mitigations
RQ2: How well do tools support the lifecycle?	Only IriusRisk (commercial); all open-source tools weak after identification	Chapter 5
RQ3: What is the LLM value?	High for identification breadth and privacy; limited for sovereignty on its own	Chapter 3, PoC results
RQ4: How to incorporate sovereignty?	Combined taxonomy + GDPR/Gaia-X control mapping	Chapter 6, Chapter 7
RQ5: Best tool or combination?	pytm + LINDDUN-LLM + PoC sovereignty layer	Chapter 5, this chapter

RQ5 needs a short explanation because the final answer is not only one tool. From the evaluation, it is clear that none of the five analysed tools can cover all requirements by itself. For teams which already use infrastructure as code, pytm would be a suitable choice because it can be integrated into a CI/CD pipeline. For the privacy part, the LINDDUN-LLM hybrid described in Section 3 is more useful, because the other evaluated tools do not really support LINDDUN in a complete way. On top of these tools, a sovereignty layer similar to the PoC developed in this thesis could be added. This is needed because the sovereignty aspect is mostly not considered by the existing tools.

8.3 Recommended Workflow

Combining the comparison results and the PoC outcomes, the practical recommendation for the proposal can be given as follows:

1. **System modelling:** first describe the cloud architecture in text form, and if needed create a DFD diagram to show the main components and connections.

2. **Identification:** use an LLM-assisted approach to find possible threats by using the combined taxonomy of STRIDE, LINDDUN, and sovereignty aspects together, instead of checking them separately.
3. **Categorisation and prioritisation:** classify the identified threats with the taxonomy and calculate their risk scores. Threats with a score of 7 or higher are considered as high priority.
4. **Mitigation:** for the high priority threats, select suitable security controls which can reduce the possible impact or probability.
5. **Reporting:** prepare a report where the detected threats and the proposed mitigations are explained for the stakeholders.
6. **Continuous re-evaluation:** perform the assessment again when important changes are made in the architecture, or otherwise repeat it regularly, for example every three months.

This is not only a single exercise, because it should be repeated after important changes in the architecture, or at least every few months. This helps to keep the process useful, because cloud environments can change over time even if there is no clear modification done by the users.

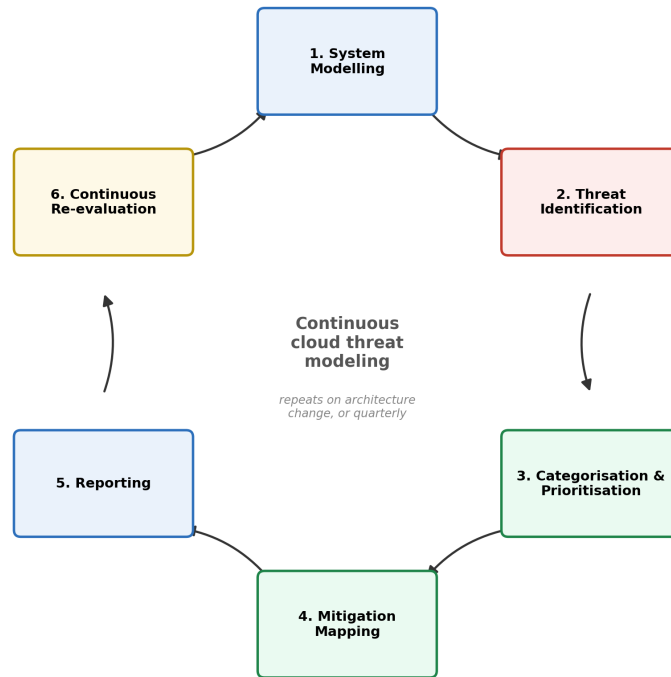


Figure 8.1: Recommended cloud threat modeling workflow, shown as a continuous cycle rather than a straight line, matching the lifecycle model introduced in Chapter 4.

Unlike the four-stage PoC lifecycle in Chapter 4, this workflow also includes system modelling and reporting steps before and after the PoC process. These steps are added because in a real project the team usually does more work than only running the tool.

8.4 Limitations

There are also some limitations for the whole thesis, not only the PoC-related ones discussed in Chapter 7:

- The comparative analysis in Chapter 5 mainly uses documentation and published papers. The five tools were not all installed and tested on the same system, so the comparison is not based on direct experiments.
- The sovereignty taxonomy was created from primary legal sources. However, it was only developed in this thesis and was not reviewed by external experts. It was also not validated or standardised by organisations like ENISA.
- The PoC was tested only with one simplified system description. Real production

systems are usually more complex, so it is not clear if the same pipeline would work in the same way for these systems.

- Since the solution uses an LLM, some non-deterministic behaviour cannot be avoided. So, the results in Chapter 7 may be a little different if the experiment is executed again.

8.5 Chapter Summary

The PoC fulfilled nine from the ten requirements. Requirement R07 is marked as Partial, because there is a specific reason for it and not because it was unclear. All five research questions can be answered based on the comparison analysis or from the PoC results. The workflow that is recommended uses different existing tools together, since one tool cannot cover everything. Also, the limitations described above show where the evidence in this thesis is still not strong enough and where more work would be useful.

Chapter 9

Conclusion and Future Work

9.1 Summary of Contributions

Looking back at this thesis, I think it produced mainly four things.

A comparative analysis that includes sovereignty. I compared five threat modeling tools and approaches using four different dimensions. From what I found in the literature review in Chapter 3, sovereignty was not used before as one comparison dimension, at least I could not find a published work that included it. The comparison showed that no existing open-source tool supports the complete lifecycle together with privacy and sovereignty. Also, I did not find any tool that really tries to address sovereignty as part of the threat modeling process.

A combined threat taxonomy. STRIDE, LINDDUN, and one sovereignty category are combined into one classification scheme. The sovereignty category is based on GDPR Chapter V, the US CLOUD Act, and the Gaia-X Trust Framework. This taxonomy is used to answer RQ4. Instead of only discussing sovereignty as a general idea, it puts sovereignty together with security and privacy in one classification. Like this, the threats can be checked and compared more easily.

A working Proof-of-Concept. A Python pipeline was implemented to run the combined taxonomy from start to end with a plain-text system description. It uses a locally hosted LLM only to identify the threats, while all later processing is done with deterministic Python code. In one example system, the pipeline detected 10 threats. Three of them were sovereignty-related threats, and these were not detected by the evaluated tools.

A reference workflow. A practical workflow with six steps (Section 8) that can be used again and again, not only one time. It is based on the results from the comparison and the PoC, so it comes from what was actually observed, not only from abstract ideas.

9.2 Answers to Research Questions

Going back again to the five questions presented in Chapter 1:

- CIS Benchmarks and the CSA Cloud Controls Matrix provide the most practical controls for cloud systems. NIST SP 800-53 is also useful, but it is more general and not focused only on cloud environments.
- Most existing tools work quite well for identifying threats, but they are much weaker in the later stages. IriusRisk is different because it supports almost the full lifecycle, however it is a commercial solution.
- LLMs can help to find more threats, especially for privacy analysis. Still, the results should be checked by a human. Also, without a taxonomy like the one developed in this thesis, the model usually does not consider sovereignty issues by itself.
- Sovereignty threats can be added in a more systematic way if there is a taxonomy and a mapping to existing security controls. This thesis includes both of these parts.
- There is no single tool that is enough for every case. A better solution is to combine different approaches, for example pytm for CI/CD integration, a LINDDUN-LLM approach for privacy analysis, and an additional sovereignty layer similar to the PoC presented in this thesis.

9.3 Future Work

There are still quite a few things that are not finished here. The five points below show specific missing parts. They are not just saying that "more research is needed."

Testing this on something real. Everything in Chapter 7 is tested with one example that I implemented by myself. It is a clean example and easy to control, but it is still not a real production system. For this reason, the next step should be testing the approach on a real architecture. It would be better if an organisation allows a researcher to access their actual system, even if the setup has many problems or is not perfectly organised.

Getting the sovereignty taxonomy checked by other people. I made this taxonomy by reading primary sources like GDPR and different court decisions. Still, I am only one bachelor's student and I am not a lawyer or part of any standards organization. Because of this, it would be useful if ENISA, the BSI, or the Gaia-X community could review it. They may confirm that the taxonomy is correct, or they may find parts that should be changed. This kind of feedback would be much more valuable than only checking it by myself.

A better model, or a better-grounded one. In the PoC, only 2 out of 4 targeted LINDDUN sub-categories were detected with the CPU-based Llama 3 model I could

use. This is probably because the model was limited and also running only on CPU. Using a bigger model, or a model fine-tuned for this task, could improve the detection. Another possibility is a retrieval-augmented approach, where the LLM can use recent threat intelligence instead of depending only on the knowledge from its training. This would maybe help to detect the missing categories better.

Actually using this in a real tool. At the moment, the taxonomy and workflow only exist as a standalone script. It would be more useful if it is implemented as a plugin for a tool that people already use. OWASP Threat Dragon and pytm are probably the best choices because both of them are open-source. This can help more people to use it, compared with keeping only a script inside the thesis repository.

Multi-cloud. Everything in this work was tested only with one cloud provider, which is AWS. In real organisations, many systems are deployed on more than one cloud provider. Because of that, the trust boundaries become more difficult to manage, and also legal or jurisdiction issues can be different between providers. These situations were not considered in this work and would need more investigation.

9.4 Closing Remarks

If there is one main idea I want the reader to remember from this thesis, it is that looking at security, privacy and sovereignty as three different tasks is not only inconvenient, but it can also make some real threats not be noticed. The example in Chapter 6 showed this with a normal signup form, which at first looks very simple. In my opinion, these three topics are connected much more than they are usually treated. They all ask almost the same question, which is how much control an organisation still has over its own data and systems after the data is outside the organisation.

The taxonomy and PoC in this thesis do not solve this problem completely. They mainly show that the problem can be solved. By combining these three different views in one analysis pass, it is possible to detect issues with relatively simple tools. During this work, I also found some threats that the existing tools I tested did not detect.

Bibliography

- [1] P. Mell and T. Grance, “The nist definition of cloud computing,” National Institute of Standards and Technology, Tech. Rep. SP 800-145, 2011. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-145>.
- [2] European Parliament and Council of the European Union, *Regulation (eu) 2016/679 (general data protection regulation)*, 2016. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>.
- [3] U.S. Congress, *Clarifying lawful overseas use of data act (cloud act)*, 2018. [Online]. Available: <https://www.congress.gov/bill/115th-congress/house-bill/4943/text>.
- [4] Microsoft, *Shared responsibility in the cloud*, Microsoft Azure Documentation, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>.
- [5] Cloud Native Computing Foundation, *CNCF Cloud Native Definition v1.0*, <https://github.com/cncf/toc/blob/main/DEFINITION.md>, GitHub, 2018. [Online]. Available: <https://github.com/cncf/toc/blob/main/DEFINITION.md>.
- [6] OWASP Foundation, *Kubernetes security cheat sheet*, 2022. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Kubernetes_Security_Cheat_Sheet.html.
- [7] M. Souppaya, J. Morello, and K. Scarfone, “Application container security guide,” National Institute of Standards and Technology, Tech. Rep. SP 800-190, 2017. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-190>.
- [8] Amazon Web Services, *Shared responsibility model*, 2023. [Online]. Available: <https://aws.amazon.com/compliance/shared-responsibility-model/>.
- [9] Office of the Comptroller of the Currency, *Capital one fined \$80 million by federal bank regulator for 2019 data breach*, 2020. [Online]. Available: [https://www.occ.gov/news-issuances/news-releases/2020/nr-occ-2020-101.html](https://www OCC.gov/news-issuances/news-releases/2020/nr-occ-2020-101.html).
- [10] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014. [Online]. Available: <https://www.wiley.com/en-us/Threat+Modeling:+Designing+for+Security-p-9781118809990>.

- [11] Microsoft, *Microsoft threat modeling tool documentation*, 2022. [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool>.
- [12] B. Schneier, “Attack trees,” *Dr. Dobbs’s Journal*, vol. 24, no. 12, pp. 21–29, 1999. [Online]. Available: https://www.schneier.com/academic/archives/1999/12/attack_trees.html.
- [13] OWASP Foundation, *Attack surface analysis cheat sheet*, 2021. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.html.
- [14] G. Stoneburner, A. Goguen, and A. Feringa, “Risk management guide for information technology systems,” National Institute of Standards and Technology, Tech. Rep. SP 800-30, 2002. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-30>.
- [15] FIRST.org, *Common vulnerability scoring system v3.1: Specification document*, 2019. [Online]. Available: <https://www.first.org/cvss/v3.1/specification-document>.
- [16] LINDDUN, *Linddun privacy threat modeling*, <https://linddun.org>. [Online]. Available: <https://linddun.org>.
- [17] L. Sion, D. Van Landuyt, K. Yskout, and W. Joosen, “Automated threat modeling using machine learning and natural language processing,” *IEEE Access*, 2023.
- [18] ENISA, *Guidelines for smes on the security of personal data processing*, 2017. [Online]. Available: <https://www.enisa.europa.eu/publications/guidelines-for-smes-on-the-security-of-personal-data-processing>.
- [19] Court of Justice of the European Union, *Judgment in case c-311/18, data protection commissioner v facebook ireland ltd and maximillian schrems (“schrems ii”)*, 2020. [Online]. Available: <https://curia.europa.eu/juris/liste.jsf?num=C-311/18>.
- [20] ENISA, *Data protection engineering*, 2022. [Online]. Available: <https://www.enisa.europa.eu/publications/data-protection-engineering>.
- [21] Gaia-X European Association for Data and Cloud AISBL, *Gaia-x architecture document*, 2024. [Online]. Available: <https://docs.gaia-x.eu/technical-committee/architecture-document/24.04/>.
- [22] National Institute of Standards and Technology, *The nist cybersecurity framework (csf) 2.0*, 2024. [Online]. Available: <https://www.nist.gov/cyberframework>.
- [23] Center for Internet Security, *Cis benchmarks*, <https://www.cisecurity.org/cis-benchmarks>. [Online]. Available: <https://www.cisecurity.org/cis-benchmarks>.
- [24] ISO/IEC, *Iso/iec 27001:2022 information security management systems – requirements*, 2022. [Online]. Available: <https://www.iso.org/standard/27001>.

- [25] Cloud Security Alliance, *Cloud controls matrix v4.0*, 2021. [Online]. Available: <https://cloudsecurityalliance.org/research/cloud-controls-matrix>.
- [26] OWASP Foundation, *Owasp threat dragon*, <https://owasp.org/www-project-threat-dragon/>, 2023.
- [27] IriusRisk, *Iriusrisk automated threat modeling platform*, <https://www.iriusrisk.com/>, 2023.
- [28] I. Tarandach and OWASP Foundation, *Pytm: A pythonic framework for threat modeling*, <https://github.com/izar/pytm>, 2021.
- [29] ThreatSpec, *Threatspec: Continuous threat modeling through code*, <https://github.com/threatspec/threatspec>, 2018.
- [30] Faily, Shamal, *Cairis: Computer aided integration of requirements and information security*, <https://cairis.org/>, 2023.
- [31] M. A. Ferrag, A. Battah, N. Tihanyi, M. Debbah, T. Lestable, and L. C. Cordeiro, “Securityllm: A large language model for cybersecurity threat detection and response,” *arXiv preprint arXiv:2311.13773*, 2023.
- [32] OWASP Foundation, *Owasp top 10 for large language model applications*, <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2023.
- [33] OWASP Foundation, *Threat modeling process*, https://owasp.org/www-community/Threat_Modeling_Process, 2023.
- [34] Ollama, *Ollama: Run large language models locally*, <https://ollama.com>, 2024.
- [35] Meta AI, *Llama 3: Open foundation and instruction-tuned language models*, <https://ai.meta.com/blog/meta-llama-3/>, 2024.